narRater

User Tutorial

A step-by-step guide to multi-step natural-language processing

Covering setup, operation, input/output formats, and all processing methods

Table of Contents

1 Overview

narRater is a multi-step pipeline for processing natural human language -- speech, conversations, interviews, long narratives, and other multi-sentence material. It transforms raw audio or text into corrected, segmented, and scored formats suitable for research and model evaluation. The pipeline preserves original wording, meaning, and style while fixing errors, organising content, and linking segments across passages (e.g. matching a speaker's clauses to reference events).

The software provides both a web-based graphical interface (recommended) and command-line scripts. Every automated step also has a Manual mode for human editing.

The pipeline consists of six main steps:

- Step 1 -- Audio Transcription: convert audio recordings to text (narraters transcribe)
- Step 2 -- Story Event Segmentation: split the story transcript into numbered events (narraters segment)
- Step 3 -- Spell & Grammar Correction: fix errors in subject recall text, no rewriting (narraters correct)
- Step 4 -- Recall Parsing: split corrected recall into clause-level segments (narraters parse)
- Step 5 -- Recall Rating: match each recall segment to story events (narraters match)
- Step 6 -- Causal Rating: rate the causal strength of every story-event pair (narraters rate)

Each step can be run in Automatic mode (using rule-based algorithms or LLM API calls) or Manual mode (human editing through the web interface). The web interface shows input on the left and editable output on the right, side by side.

Every step is also available from the terminal via the 'narraters' command installed with the package -- e.g. 'narraters segment', 'narraters match'. Each step chapter below ends with a 'Terminal Command' section showing the exact invocation and options; 'narraters <step> --help' lists them at any time. Steps 1-2 work on the story, steps 3-5 on each subject's recall, and step 6 on the story-event list; a text-only project can skip Step 1.

2 Getting Started

2.1 Prerequisites

Before running the software, ensure you have:

- Python 3.10 or later installed
- Install the package: pip install -e . (or pip install -r requirements.txt)

The default install is deliberately minimal: it is enough to run the lightweight default method of every step plus the web UI, and pulls no multi-GB ML frameworks, so it installs fast and will not fill up your disk. Heavier methods are opt-in extras:

- [api] -- Anthropic / OpenAI clients for LLM-based methods (Steps 5, 6)
- [audio] -- Whisper / WhisperX for Step 1 audio transcription (pulls torch)
- [nlp] -- spaCy + benepar for higher-accuracy Step 2 fine/coarse segmentation
- [match] -- the rmatch embedding backend for Step 5 (pulls torch)
- [local-llm] -- transformers + torch for local Gemma via HuggingFace

Example: pip install -e ".[api]" | no account or password is ever required.

2.2 Setting Up API Keys

Several steps can use LLM models via API for higher-quality results. The software supports both Anthropic and OpenAI (GPT) models. To set up an API key:

- Anthropic: export ANTHROPIC_API_KEY='your-key-here'
- OpenAI: export OPENAI_API_KEY='your-key-here'
- Or run the setup script: bash scripts/setup_api_key.sh

API keys are read from environment variables and are never stored in code files. You can also enter the API key through the web interface's Pipeline Configuration page.

2.3 Starting the Web Interface

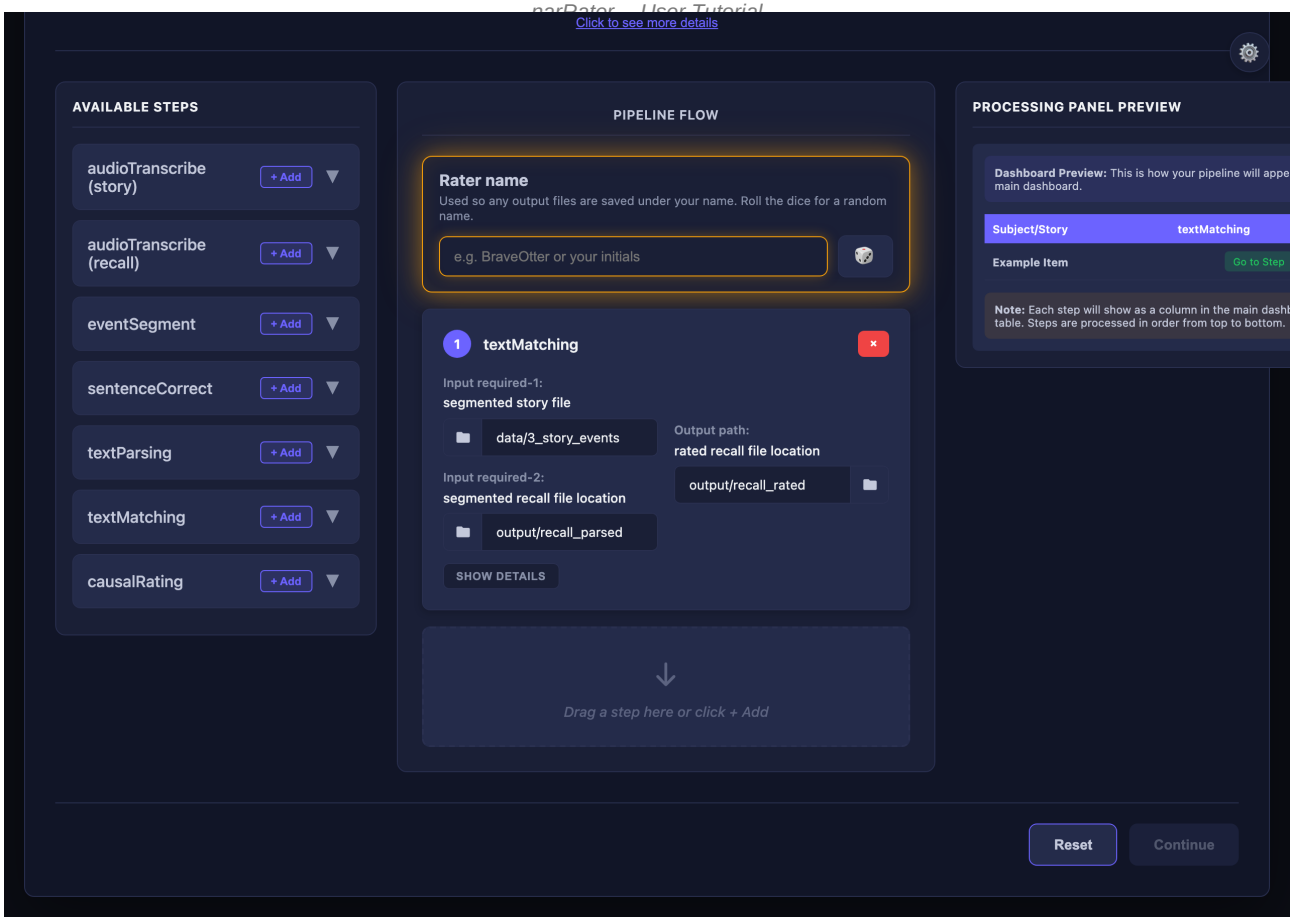
The recommended way to use the software is through its web interface. To start the server, use any one of:

- Terminal: narraters serve (auto-opens the browser)
- macOS: double-click narRater.app
- macOS: double-click server/START_HERE.command

The browser opens automatically to http://localhost:5000. If a pipeline is not yet configured you land on the Pipeline Configuration page; otherwise on the Dashboard.

2.4 Rater Name (no login)

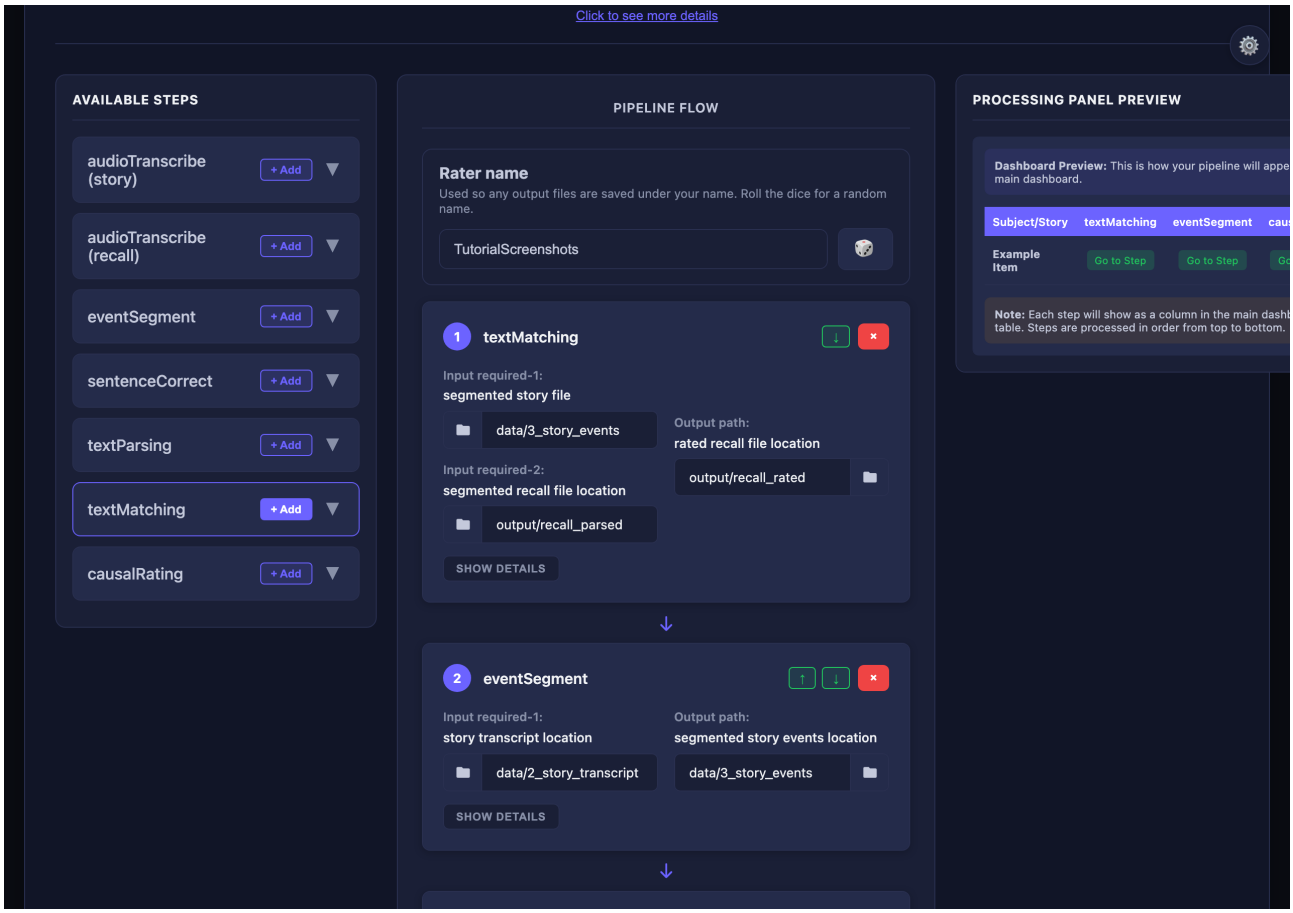
There is no login, account, or password. Instead, on the Pipeline Configuration page you enter a Rater name in the box at the top of the Pipeline Flow panel (the middle panel). This name is used only to label any output files you save by hand, e.g. the_siren_sub-01_BraveOtter-edit.xlsx -- a made-up / dummy name is perfectly fine. Click the dice button to fill in a random adjective+noun name.



the Rater name box (with the dice button) sits at the top of the Pipeline Flow panel in the middle. The Continue button stays greyed out until a rater name is entered AND

3 Pipeline Configuration

When the app starts, you are taken to the Pipeline Configuration page. Here you define which processing steps to run and in what order.



configuration builder -- drag steps from the 'Available Steps' palette on the left into the 'Pipeline Flow' canvas in the middle. The Rater name box and dice sit at the top of

The left panel ('Available Steps') is the palette. Drag steps into the 'Pipeline Flow' canvas in the middle to build your workflow. Each step has configurable input and output paths that determine where it reads data from and writes results to.

Configurable options for each step:

- Input path(s): one or more folders/files the step reads from. Steps with a single input show 'Input required'; steps that need two sources (e.g. Text Matching) show 'Input required-1' and 'Input required-2' with short labels above each path box.
- Output Path: the directory where the step writes output files
- Method: the processing method (automatic algorithm, API call, or Manual)

Text Matching is the main two-input step: 'Input required-1' is the segmented reference (story events) and 'Input required-2' is the parsed recall/segment file location. Use the folder picker (Browse) on any path; shortcuts such as Home and Desktop jump to common locations.

At the top of the Pipeline Flow panel, enter a Rater name (or click the dice for a random one). The 'Continue' button is greyed out and not clickable until BOTH a rater name is entered AND at least one step has been added. Clicking 'Continue' saves the configuration to pipeline_config.json and opens the main dashboard; you can return to this page anytime via the 'Change Rater' button on the dashboard.

4 Main Dashboard

The dashboard shows all subjects and stories detected from your data directories, grouped into one panel per pipeline chain. Each row is a subject/story and each column is a pipeline step; every cell shows that step's status for that item. Click an item to open its detail view.

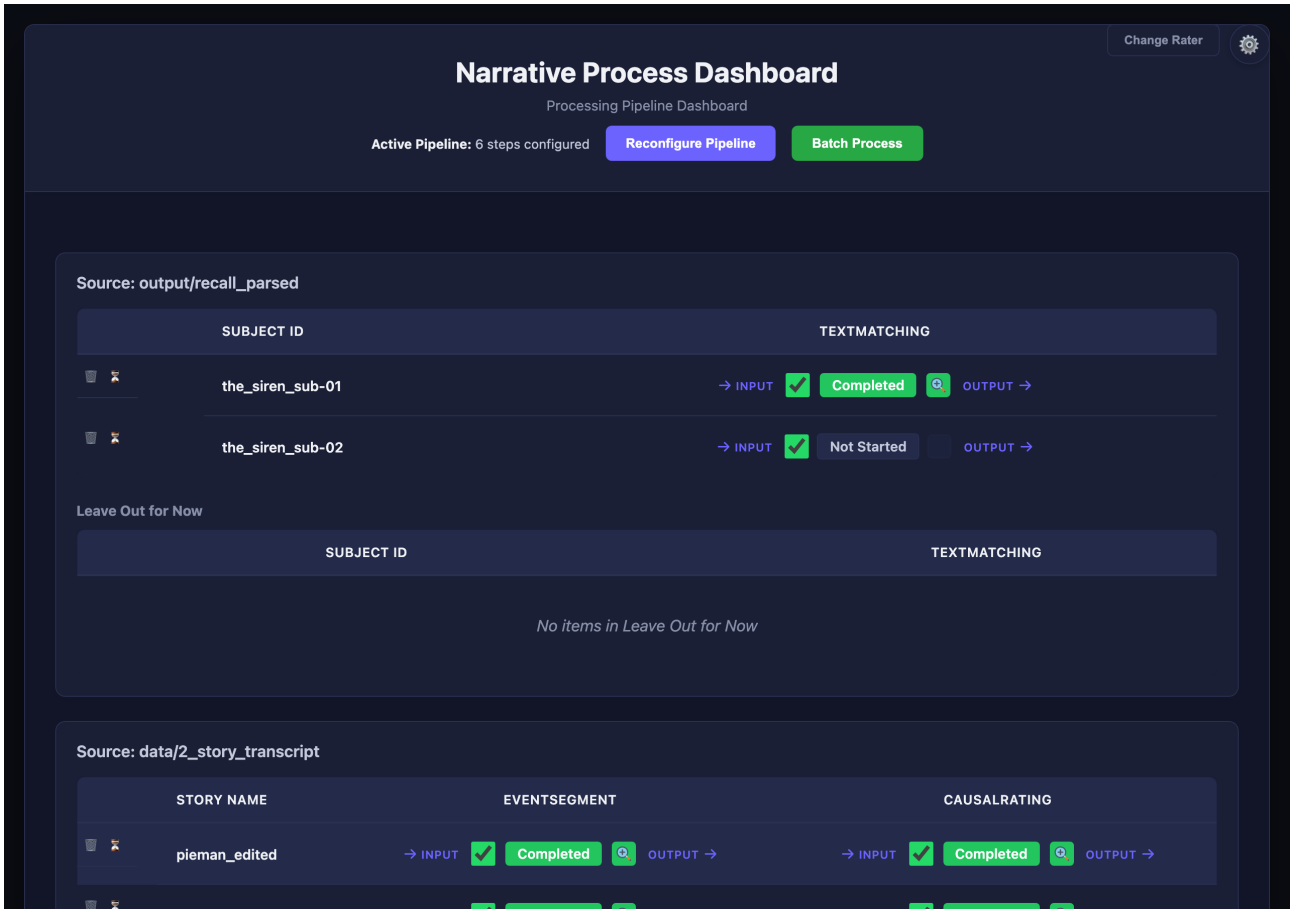


Figure 3: Main dashboard -- one panel per pipeline chain; rows are subjects/stories, columns are steps. Click a step cell to auto-process it for that item.

Status colours: green = completed, yellow = in progress, grey = not started. To AUTO-PROCESS a step for one item, click its cell. For steps that have more than one method, a dialog opens where you choose the Method, Model, Prompt version, and Input variant before it runs. Batch buttons run a step across all items at once.

The File Version dropdown switches between the automated output and any {ratername}-edit versions when both exist. The 'Change Rater' button (top right) returns to the Pipeline Configuration page so you can change the rater name or the pipeline.

4.1 Heavy-Method Warning

Some methods load multi-GB local models -- Gemma-4 via Ollama, the rMatch embedding matcher, local Transformers, or Whisper. Before launching one, the app runs a resource preflight that checks available RAM, free disk space, and whether the model / runtime is even installed. It never downloads or starts a model to make this decision, so the check itself cannot crash the machine.

If the chosen method is likely too heavy for your device, a popup appears before anything is spawned. It explains why (e.g. not enough free RAM, not enough disk for the model) and offers three choices:

- Use lighter method -- one click switches to a free, no-model method (e.g. rules, keyword 'test', clause) and runs that instead

- Run anyway -- proceed with the heavy method despite the warning
- Cancel -- do nothing

On a capable machine with the model already installed, the checks pass silently and no popup interrupts the run.

5 Input Files and Formats

5.1 Story Transcripts

Location: data/2_story_transcript/

Format: Plain text files (.txt), one file per story

Naming: {story_name}.txt

The story transcript is the full text of the narrative to be segmented into events.

Example: Story transcript file (my_story.txt)

```
Once upon a time there was a young girl named Clara.
She lived in a small town near the mountains.
One day, she decided to go on an adventure...
```

5.2 Recall Text Files

Location: data/5_recall_texts/

Format: Plain text files (.txt), one file per subject

Naming: {story_name}_sub-{subject_id}.txt

Each file contains one subject's recall of the story, transcribed verbatim.

Example: Recall text file (my_story_sub-001.txt)

```
there was a girl named clara who lived in a mountain town
she went on an adventure one day and met a wise old man
```

5.3 Audio Files (Optional)

Story audio: data/1_story_audio/ (.wav, .mp3, .mp4, .m4a, .flac, .ogg)

Recall audio: data/4_recall_audio/ (same formats)

Audio files are used for automatic transcription (Step 1). If you already have text transcripts, audio files are not required.

5.4 Summary Excel Files (Alternative)

Location: data/summary_*.xlsx (one file per condition)

Required columns: 'sub' (subject number), 'ID' (subject ID), 'recall' (recall text)

These Excel files can be used as an alternative source for recall text. The software will extract individual subject recalls from the 'all' sheet.

Example: Summary Excel structure

```
Sheet: 'all'
| sub | ID   | recall                              |
|-----|------|-------------------------------------|
| 1   | 1001 | there was a girl named clara who... |
| 2   | 1002 | clara lived near mountains and...   |
```

5.5 Flexible Tabular Formats

Story events, parsed segments, and rated outputs are not limited to .xlsx. The pipeline also accepts .csv, .tsv, .xls, and plain .txt where appropriate. Column headers are detected flexibly -- e.g. event number columns may be named 'event',

'event_number', or '#', and text columns may be 'story_texts', 'text', or 'segment'. For recall/rating files, 'recalled_events' and 'recall_in_temporal_order' are preferred, but common synonyms are recognised; swapped columns are corrected automatically on read and write.

In the Pipeline Configuration page you point each step at a folder; the software finds matching files by subject/story id and extension. You can mix formats across steps as long as required columns are present.

6 Step 1: Audio Transcription

6.1 What This Step Does

Converts audio recordings (story narrations or subject recall recordings) into text transcripts. This is the entry point if your data starts as audio files rather than text files.

6.2 How It Works

Input: data/1_story_audio/ or data/4_recall_audio/

Output: output/story_audio-transcribed/ or output/recall_audio-transcribed/

Available methods:

- Automatic (WhisperX / Whisper): The preferred engine is WhisperX (batched inference with word-level alignment); falls back to standard Whisper if WhisperX is not installed. Default model size: 'large-v2' (WhisperX) or 'medium' (Whisper). Performs verbatim transcription in English.
- Manual: Creates an empty transcript file. Open the Audio Transcription tab in the detail view to listen to the audio and type or paste the transcription.

6.3 Technical Details

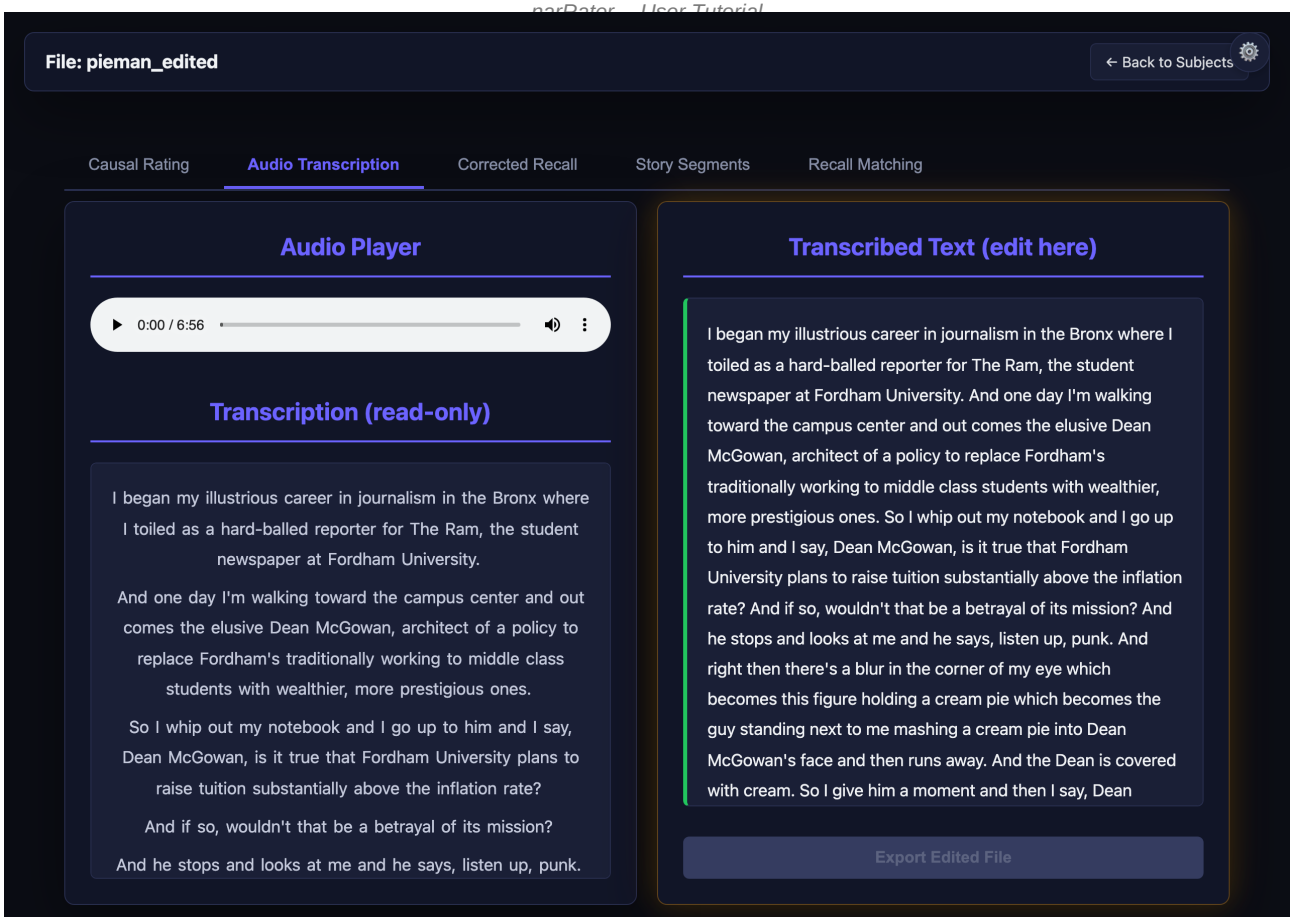
Whisper/WhisperX model sizes and their trade-offs:

- 'tiny' / 'base': Fastest, lowest accuracy
- 'small' / 'medium': Good balance of speed and accuracy
- 'large-v2' (default for WhisperX): Best accuracy, requires more memory

No API key is needed for local transcription. The model runs on your machine. Install with: pip install whisperx (or pip install openai-whisper for standard Whisper).

6.4 Interface Walkthrough

In the detail view, the Audio Transcription tab shows an audio player on the left and an editable text area on the right. You can play the audio, pause, and type the transcription. Click 'Export Edited File' to save.



inspection using bundled story-level audio inputs (same tab layout appears for recall audio with step 3 subject files). If this image is missing, run developer/capture_tuto

Example: Transcription output (my_story_sub-001.txt)

```
there was a girl named clara who lived in a mountain
town she went on an adventure one day
```

6.5 Methods & Options Reference

Method	What it does	Args / Keys
Whisper local (only)	Loads OpenAI Whisper locally (no cloud). Pick a model size from the launcher.	`--model tiny | base | small | medium | large-v2` (default: base)

6.6 Terminal Command

Step 1 also runs headless via the 'narraters' command (no GUI):

```
narraters transcribe                      # recall audio (default)
narraters transcribe --kind story --model small
narraters transcribe -i path/in -o path/out --timestamps
narraters transcribe --filter sub-01      # one item only
```

Options: --model (tiny|base|small|medium|large-v2|large-v3); --timestamps (also write a word-level timing Excel); --kind recall|story (chooses the conventional input/output directories); -i/--input and -o/--output (override those directories); --filter (process one item id). Requires the [audio] extra.

7 Step 2: Story Event Segmentation

7.1 What This Step Does

Splits a story transcript into numbered events. Each event represents a discrete unit of the narrative (an action, state change, or dialogue turn). The segmentation level determines how finely the story is divided.

7.2 How It Works

Input: data/2_story_transcript/{story_name}.txt

Output: data/3_story_events/{story_name}_events-{method}.xlsx

Output columns: 'event' (number), 'story_texts' (segment text)

The granularity hierarchy from finest to broadest is:

- Clause Level -- one independent clause per segment (finest, the atomic unit)
- Fine-Grained -- one discrete event per segment (default); each event is 1 or more consecutive clause segments
- Coarse-Grained -- one scene per segment (broadest); each scene is 1 or more consecutive fine-grained events

Each level is built compositionally on the one below it. The clause-level method produces the atomic segments. Fine-grained segmentation runs clause-level first, then merges consecutive clause segments into events. Coarse-grained segmentation runs fine-grained first, then merges consecutive fine-grained events into scene blocks. This guarantees that every boundary at a coarser level always coincides with a boundary at the finer level.

7.3 Fundamental Rule

Every segment at every level must contain at least one independent clause (a clause with a subject and a finite verb that can stand alone as a sentence). Fragments, interjections, and bare noun phrases are automatically merged with neighbouring segments. Dialogue speech acts (quoted text) are exempt from this rule.

7.4 Method: Clause Level (Atomic Unit)

The finest granularity. This is the atomic unit of segmentation -- all other non-API methods build on these segments. Splits at every independent clause boundary:

- Sentence boundaries (. ! ? followed by uppercase)
- Semicolons (always separate independent clauses)
- Non-restrictive relative clauses: ', who/which/whom'
- Speech attribution + direct speech boundaries
- Commas between independent clauses (subject + verb on both sides)
- Coordinating conjunctions (and/but/so/yet) between independent clauses

Never splits: subordinate + main clause, time/place + action, participial continuations.

Post-processing: a proof-check pass merges unjustified fragments (bare time phrases, dangling subordinates, trailing conjunctions). An independent-clause validation pass then merges any remaining non-IC segments.

7.5 Method: Fine-Grained (Default)

Each discrete action, state change, or dialogue turn is its own event. Built on clause-level output: runs clause-level segmentation first, then merges consecutive clause segments into events using the following rules:

- Sentence boundaries always start a new event (clauses from different sentences are never merged)
- Within a sentence, all clauses are merged into one event by default
- Temporal transitions (and then / eventually / later / suddenly / next / finally) keep the clause split when the current group has 5 or more words
- Contrast transitions (but) keep the clause split when the current group has 8 or more words
- Very short adjacent events (both 3 words or fewer) are merged together

7.6 Method: Coarse-Grained

Scene-level segmentation. Built on fine-grained output: runs fine-grained segmentation first, then merges consecutive fine-grained events into scene blocks using the following rules:

- Strong narrative transitions (Later, Eventually, Meanwhile, Suddenly, The next day, Back at, etc.) start a new scene when the current block has 15 or more words
- Forced split when a block exceeds ~100 words
- All other fine-grained events accumulate into the current scene block

7.7 Method: API (LLM Call)

Uses a language model with a prompt template to segment the story. This method sends the full story transcript to an LLM and receives back a segmented version.

Available API models:

- Sonnet 4.5 tier (Anthropic) -- requires ANTHROPIC_API_KEY
- Haiku 3.5 tier (Anthropic) -- faster, lower cost
- GPT-4o (OpenAI) -- requires OPENAI_API_KEY
- GPT-4o Mini (OpenAI) -- faster, lower cost
- Gemma 4 E4B (Ollama, local) -- no cloud key; requires Ollama with e.g. gemma4:e4b pulled
- Optional Llama tags via the same Ollama API path (see UI presets)

Select the model from the Method dropdown in the web interface. The model choice determines both quality and cost.

Prompt used (from scripts/prompt/event_segment.txt):

```
An event is an ongoing coherent situation. The following
story needs to be copied and segmented into events.
Copy the following story word-for-word and start a new
line whenever one event ends and another begins.
This is the story:
```

To modify the prompt: edit scripts/prompt/event_segment.txt directly. You can also create alternate versions (e.g., event_segment_v2.txt) and select them from the command line with --prompt-version.

7.8 Method: Manual

Places the full transcript as a single event. Open the Story Segments tab in the detail view to manually split/merge events using the Split and Merge buttons.

7.9 Interface Walkthrough

In the detail view, the Story Segments tab shows the full story transcript on the left and the segmented events on the right. Each event is shown in an editable text box with its event number.

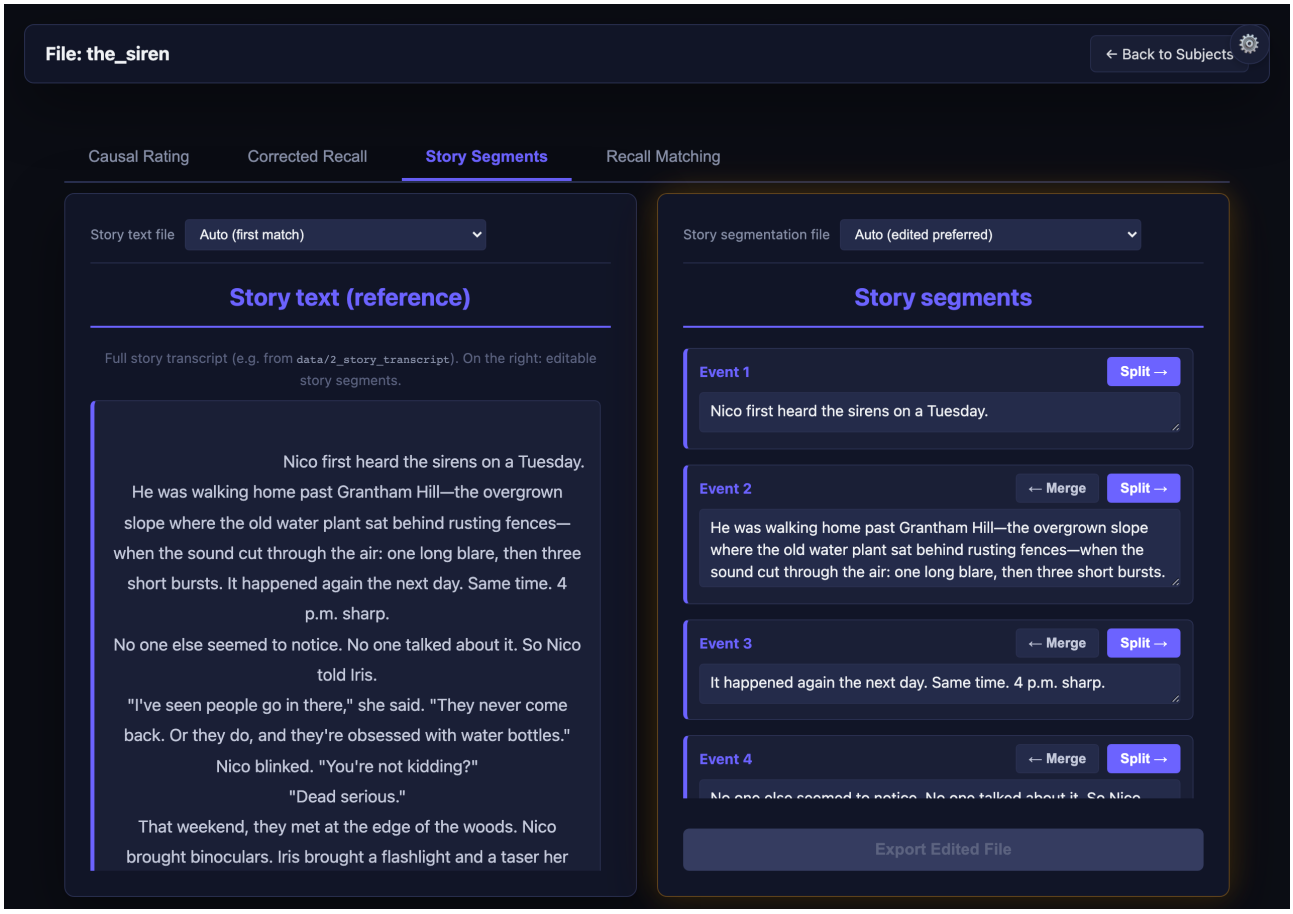


Figure 4: Story events segmentation view -- left: full transcript; right: segmented events

7.10 Methods & Options Reference

Method	What it does	Args / Keys
clause	Rule-based atomic-clause segmentation. Uses spaCy if installed, else regex.	`no key needed`
fine (default)	Rule-based mid-grained segmentation. Merges adjacent clauses into events.	`no key needed`
coarse	Rule-based coarse segmentation. Larger event units (paragraph-like).	`no key needed`
api	LLM-based segmentation using a remote API or local Ollama.	`--model <id>` `--prompt-version <name>` `ANTHROPIC_API_KEY` or `OPENAI_API_KEY` (none for Ollama)
manual	Places the full transcript as a single event for hand-segmentation in the UI.	`no key needed`

Models offered when method=api (see Section 15.1 for the full list):

Method	What it does	Args / Keys

claude-haiku-3-5-20241022

Haiku 3.5 (Anthropic); see Anthropic console for pricing.

		ANTHROPIC_API_KEY
claude-haiku-4-5-20251001	Haiku 4.5 (Anthropic); see Anthropic console for pricing.	ANTHROPIC_API_KEY
claude-opus-4-7	Opus 4.7 (Anthropic); see Anthropic console for pricing.	ANTHROPIC_API_KEY
claude-sonnet-4-5-20250929	Sonnet 4.5 (Anthropic); see Anthropic console for pricing.	ANTHROPIC_API_KEY
claude-sonnet-4-6	Sonnet 4.6 (Anthropic); see Anthropic console for pricing.	ANTHROPIC_API_KEY
gpt-4o / gpt-4o-mini	OpenAI alternatives.	OPENAI_API_KEY
gemma4-e4b-ollama	Local Gemma 4 E4B via Ollama; no cloud key.	EVENT_SEGMENT_OLLAMA_MODEL override optional
llama3.3-ollama / llama5.3-ollama	Local Llama tags via Ollama.	same; override tag via env

Editing controls:

- Split: click between two events to insert a boundary
- Merge: combine adjacent events into one
- Export Edited File: save your edits as {story}_events_{ratername}-edit.xlsx

Example: Story events output ({story_name}_events-fine.xlsx)

```
| event | story_texts                                    |
|-------|------------------------------------------------|
| 1     | Once upon a time there was a young girl Clara.  |
| 2     | She lived in a small town near the mountains.   |
| 3     | One day, she decided to go on an adventure.     |
```

7.11 Terminal Command

Run Step 2 from the terminal:

```
narraters segment --method clause
narraters segment --method fine -i data/2_story_transcript/my_story.txt
narraters segment --method api --model claude-sonnet-4-6 \
                  --prompt-version event_segment
narraters segment --list-prompts     # show available prompt versions
```

--method: clause | fine | coarse | api. --model is used only with --method api. -i/--input may be a single transcript file or a directory; -o/--output defaults to data/3_story_events/.

8 Step 3: Spell & Grammar Correction

8.1 What This Step Does

Corrects spelling and grammar errors in recall text while strictly preserving the original meaning, structure, and wording. Only clear errors are fixed -- the step never rewrites, paraphrases, or restructures sentences.

8.2 How It Works

Input: data/5_recall_texts/{subject_id}.txt (or summary Excel)

Output: output/recall_corrected/{subject_id}.txt (rules), or Gemma outputs *_spell-gemma-hf_* / *_spell-ollama_*

Available methods:

- Automatic (Rule-Based, default): A multi-pass pipeline that applies conservative corrections in sequence.
- Manual: Copies the raw input text to the output folder. Open the Corrected Recall tab in the detail view to manually correct the text side-by-side with the original.
- Gemma 4 (Hugging Face, local): Uses google/gemma-4-E4B-it by default with scripts/prompt/spell_gram.txt. Requires PyTorch + transformers.
- Gemma 4 (Ollama, local): Same prompt via Ollama (default model tag gemma4:e4b); optional override in the UI.

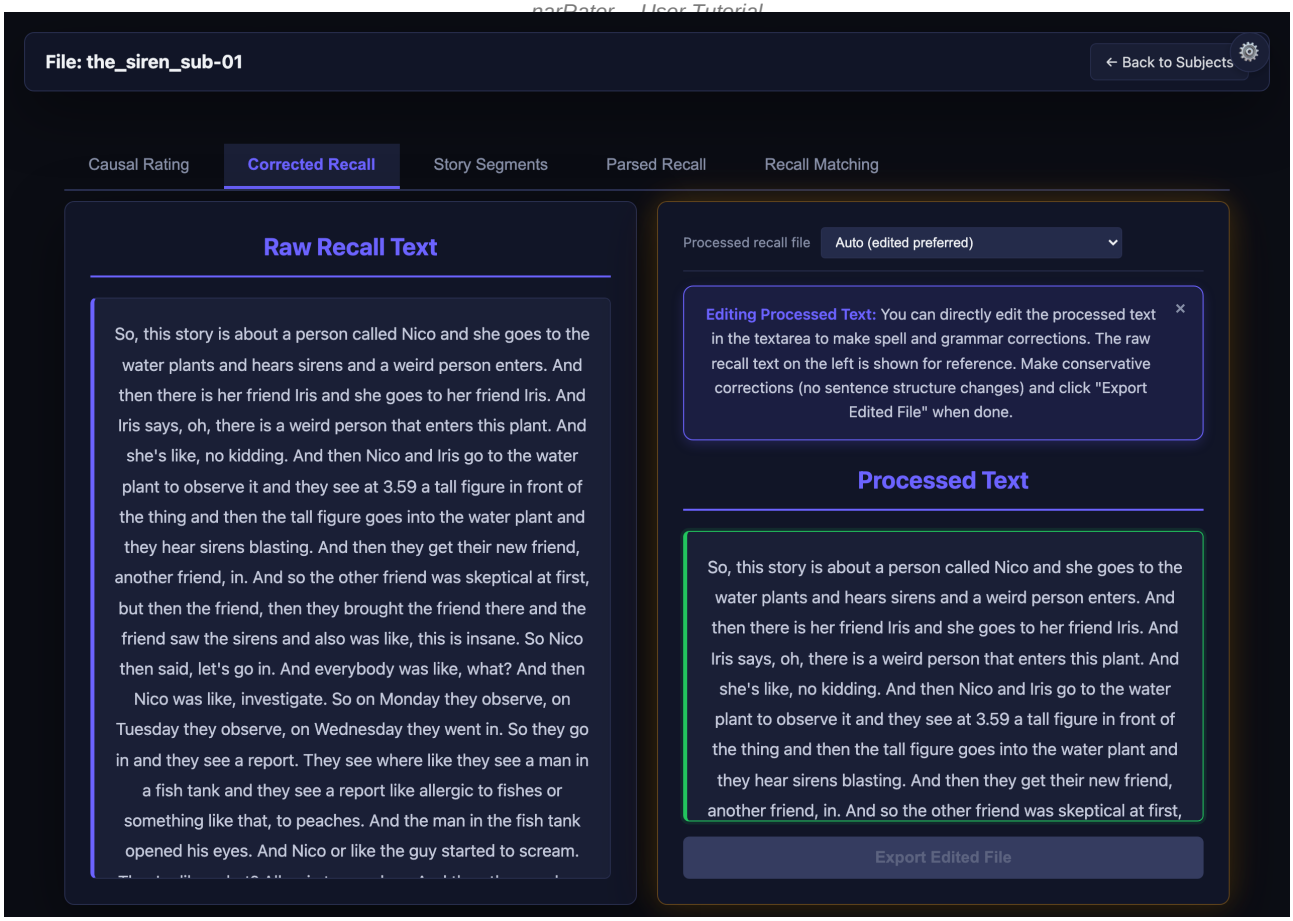
8.3 Automatic Processing Pipeline

The automatic mode applies corrections in this order:

- 1. Abbreviation protection: protects known abbreviations (p.m., a.m., e.g., i.e., etc.) from incorrect splitting
- 2. Contraction repair: fixes broken contractions (could't -> couldn't, did'nt -> didn't)
- 3. Grammar fixes: corrects common grammar issues ('i' -> 'I', 'i a,' -> 'I am,')
- 4. Capitalisation: adds capitals after sentence-ending punctuation (not after abbreviation periods)
- 5. Common misspellings: fixes frequent misspellings (seperate -> separate, recieve -> receive)
- 6. Punctuation cleanup: fixes spacing, duplicates, misplaced periods
- 7. Spell-check: uses pyspellchecker with a whitelist for proper nouns, acronyms, Roman numerals. Blocks plural-to-singular and adjective-to-verb changes.
- 8. LanguageTool (optional): additional grammar fixes, skipping style/typography rules

8.4 Interface Walkthrough

In the detail view, the Corrected Recall tab shows the raw recall text on the left and the corrected text on the right. You can directly edit the corrected text and save it.



Subject inspection -- Corrected Recall tab (Step 3).

Example: Before correction

```
there was a girl named clara who lived in a mountian
tow she went on an adventure one day
```

Example: After correction

```
There was a girl named Clara who lived in a mountain
town. She went on an adventure one day.
```

8.5 Methods & Options Reference

Method	What it does	Args / Keys
rule-based (default)	Local spell-checker + targeted grammar fixes. Conservative; no rewrites.	`no key needed`
gemma-ollama	Local Gemma 4 via Ollama; same minimal-correction prompt as the cloud path.	`Ollama running locally`
manual	Copies raw text into the corrected output for hand-editing in the UI.	`no key needed`

8.6 Terminal Command

Run Step 3 from the terminal:

```
narraters correct --method rules
narraters correct --method gemma-ollama --ollama-model gemma4:e4b
```

--method: rules (entirely local, no model) | gemma-ollama (needs a local Ollama server). --ollama-model sets the model tag; --prompt-file overrides the instructions file; -i/--input is a single recall text file and -o/--output its directory.

9 Step 4: Recall Parsing

9.1 What This Step Does

Splits the corrected recall text into clause-level segments. Each segment represents one idea unit that can be matched to a story event. This step uses the same fundamental rule as story segmentation: every segment must contain at least one independent clause.

9.2 How It Works

Input: output/recall_corrected/{subject_id}.txt or method-specific spell outputs

Output: output/recall_parsed/{subject_id}_parsed.xlsx (rules) or {subject_id}_parsed-ollama_*.xlsx (Ollama)

Output columns: 'recalled_events' (empty, filled in Step 5), 'recall_in_temporal_order' (segment text)

Available methods:

- Automatic (Clause-Based, default): Uses the same clause-level splitting rules as story segmentation (sentence boundaries, semicolons, non-restrictive relative clauses, comma-separated independent clauses, coordinating conjunctions). Includes a proof-check pass and independent-clause validation.
- Manual: Places the full corrected text as a single segment. Use Split/Merge buttons in the detail view to segment manually.
- Gemma 4 (Ollama, local): Uses scripts/prompt/recall_parse_clause.txt with your local Ollama server; output filename includes the method tag.

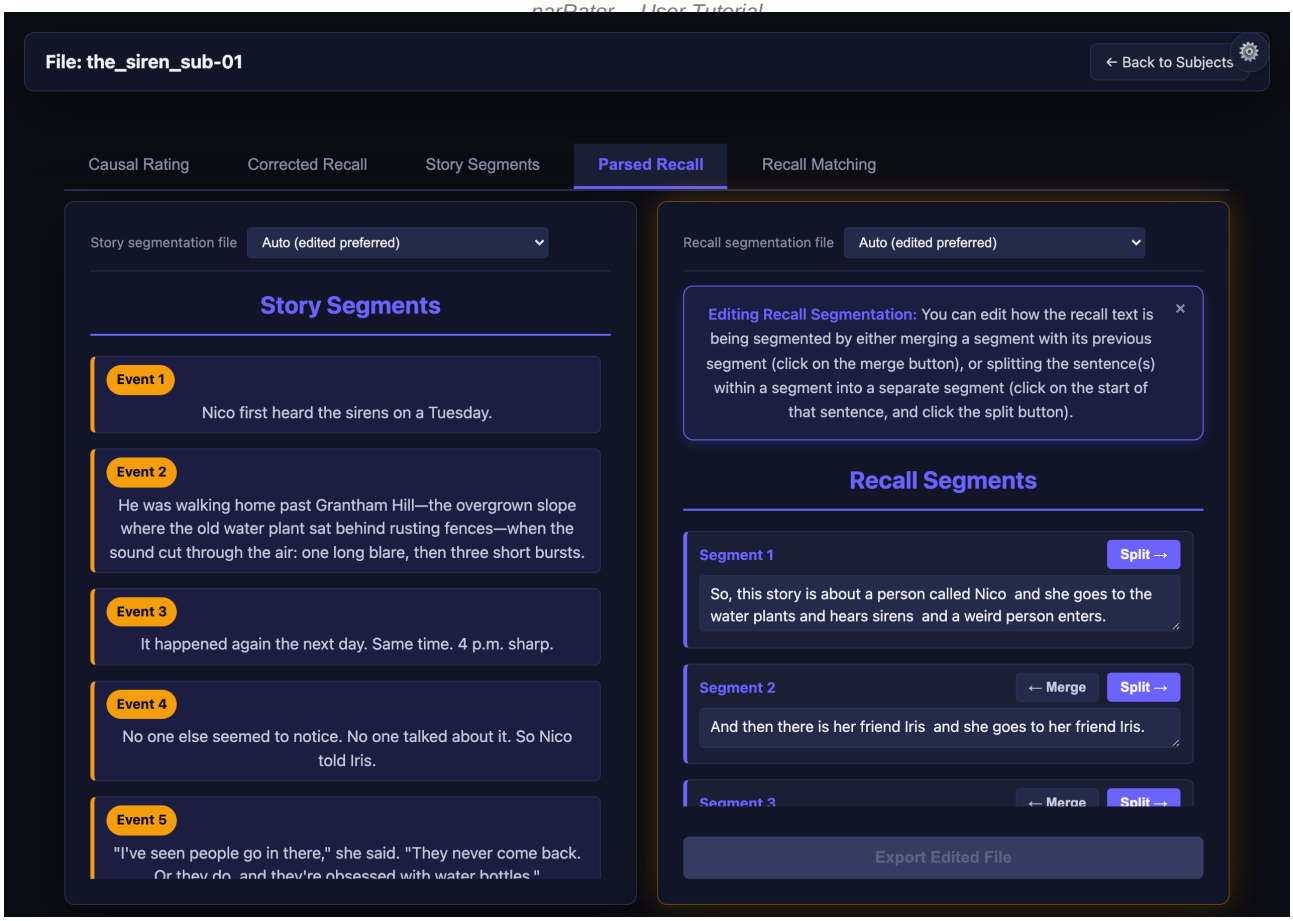
9.3 Splitting Rules (Automatic Mode)

- Sentence boundaries (. ! ? followed by capital letter, ignoring abbreviation periods)
- Closing-quote sentence boundaries (. ! ? + " correctly keeps quote with its sentence)
- Semicolons (always split)
- Non-restrictive relative clauses (, who/which/whom with own verb)
- Commas between independent clauses (>= 6 words before, new subject+verb after)
- Coordinating conjunctions (and/but/so/yet) between independent clauses with different subjects

Optional NLP enhancement: If spaCy is installed (pip install spacy && python -m spacy download en_core_web_sm), dependency parsing is used for more accurate clause boundary detection. Otherwise regex heuristics are applied.

9.4 Interface Walkthrough

In the detail view, the Parsed Recall tab shows the story events on the left (for reference) and the parsed recall segments on the right. Each segment is an editable text box.



Subject inspection -- Parsed Recall tab (Step 4).

Example: Parsed recall output ({subject_id}_parsed.xlsx)

```
| recalled_events | recall_in_temporal_order                    |
|-----------------|---------------------------------------------|
|                 | There was a girl named Clara.               |
|                 | She lived in a mountain town.               |
|                 | She went on an adventure one day.           |
```

9.5 Methods & Options Reference

Method	What it does	Args / Keys
rule-based (default)	Sentence-level splitter with regex clause boundaries. Fast and deterministic.	`no key needed`
gemma-ollama	Local Gemma 4 via Ollama for clause parsing.	`Ollama running locally` `RECALL_PARSE_METHOD=ollama`
manual	Loads the corrected text as a single segment for hand-splitting in the UI.	`no key needed`

9.6 Terminal Command

Run Step 4 from the terminal:

```
narraters parse --method rules
narraters parse --method ollama --model gemma4:e4b
narraters parse --filter-pattern sub-02     # one subject only
```

--method: rules (default, regex, no model) | ollama (local Gemma via Ollama). -i/--input defaults to output/recall_corrected/ and -o/--output to output/recall_parsed/; --filter-pattern limits to one subject.

10 Step 5: Recall Rating

10.1 What This Step Does

Matches each recall segment to one or more story events. This is the final processing step that produces the scored recall data suitable for analysis.

10.2 How It Works

Input: parsed recall/segments (e.g. output/recall_parsed/{subject_id}_parsed.xlsx) plus segmented reference events (e.g. data/3_story_events/{story}_events.xlsx). Both accept .xlsx, .csv, or .tsv; configure both paths in Pipeline Configuration ('Input required-1' / 'Input required-2').

Output: output/recall_rated/{subject_id}_rate-recall*.{xlsx|csv|tsv} (filename suffix shows API vs Ollama vs test mode). Column 1 = matched event number(s); column 2 = recall segment text.

Available methods:

- Test Mode (default): Keyword/phrase matching without API. Uses phrase overlap, concept matching, and word overlap with score thresholds. Returns up to 3 matched events per segment. Fast, no cost, no API key needed.
- API (LLM Call): Sends all recall segments and story events to an Anthropic Messages API model in a single batch request. The launcher exposes a Model dropdown (Opus 4.7, Sonnet 4.6, Haiku 4.5, Sonnet 4.5, Haiku 3.5 -- see Section 14.1). More accurate for semantic matching. Requires ANTHROPIC_API_KEY.
- Manual: Copies parsed segments with empty event-match fields. Assign story event numbers manually in the detail view.
- Gemma 4 (Ollama, local): Same batch JSON prompt as the cloud API path (scripts/prompt/recall_rating.txt) via Ollama; default tag gemma4:e4b. Output is JSON-schema-constrained so the model returns one match object per recall row even without an Anthropic key.
- rMatch (Hugging Face, local): Uses the PyPI rmatch package (github.com/gabrielkp/rmatch) with a local Hugging Face causal-LM. Best when a GPU/MPS is available. See Section 10.5.

10.3 API Method Details

When using the API method, the software sends all recall segments and story events to the Anthropic Messages API in a single batch request. The model returns a JSON mapping each recall row to its matched event number(s).

Prompt used (from scripts/prompt/recall_rating.txt):

```
You are given two Excel files:
story_events.xlsx (each row is a story event; columns:
  'event', 'story_texts') and
story_recall.xlsx (each row is one recall segment).
Go through the recall file row by row.
For each recall segment, identify which story event(s)
  it refers to.
* Write the matching event number(s) into column 1.
* If multiple events are referenced, separate with commas.
* If no event matches, use NONE.

Output format (STRICT): Output ONLY valid JSON array.
Each element: {"row_index": N, "matched_events": ...}
```

To modify the prompt: edit scripts/prompt/recall_rating.txt directly. Alternative prompt versions (recall_rating_v2.txt, recall_rating_v3.txt, etc.) can be created in the scripts/prompt/ folder and selected via the RECALL_RATING_PROMPT environment variable:

```
export RECALL_RATING_PROMPT="recall_rating_v2"
python scripts/5_recall-rater.py
```

See scripts/prompt/README.md for documentation on all available prompt versions and their differences.

10.5 rMatch Method (Hugging Face, local)

The rMatch method uses the PyPI rmatch package (github.com/gabrielkp/rmatch) to score recall-to-event matches with a locally hosted Hugging Face causal-LM. Unlike the Ollama path, rmatch runs the model directly via transformers/bitsandbytes, so it can leverage GPU/MPS when available and run any HF chat-tuned model.

Requirements:

- pip install rmatch (already listed in requirements.txt)
- Hugging Face token (HF_TOKEN) in software/.env, in environment, or via huggingface-cli login
- Enough disk + RAM for the chosen model; see warning below

Configuration via environment variables (or step options in the UI):

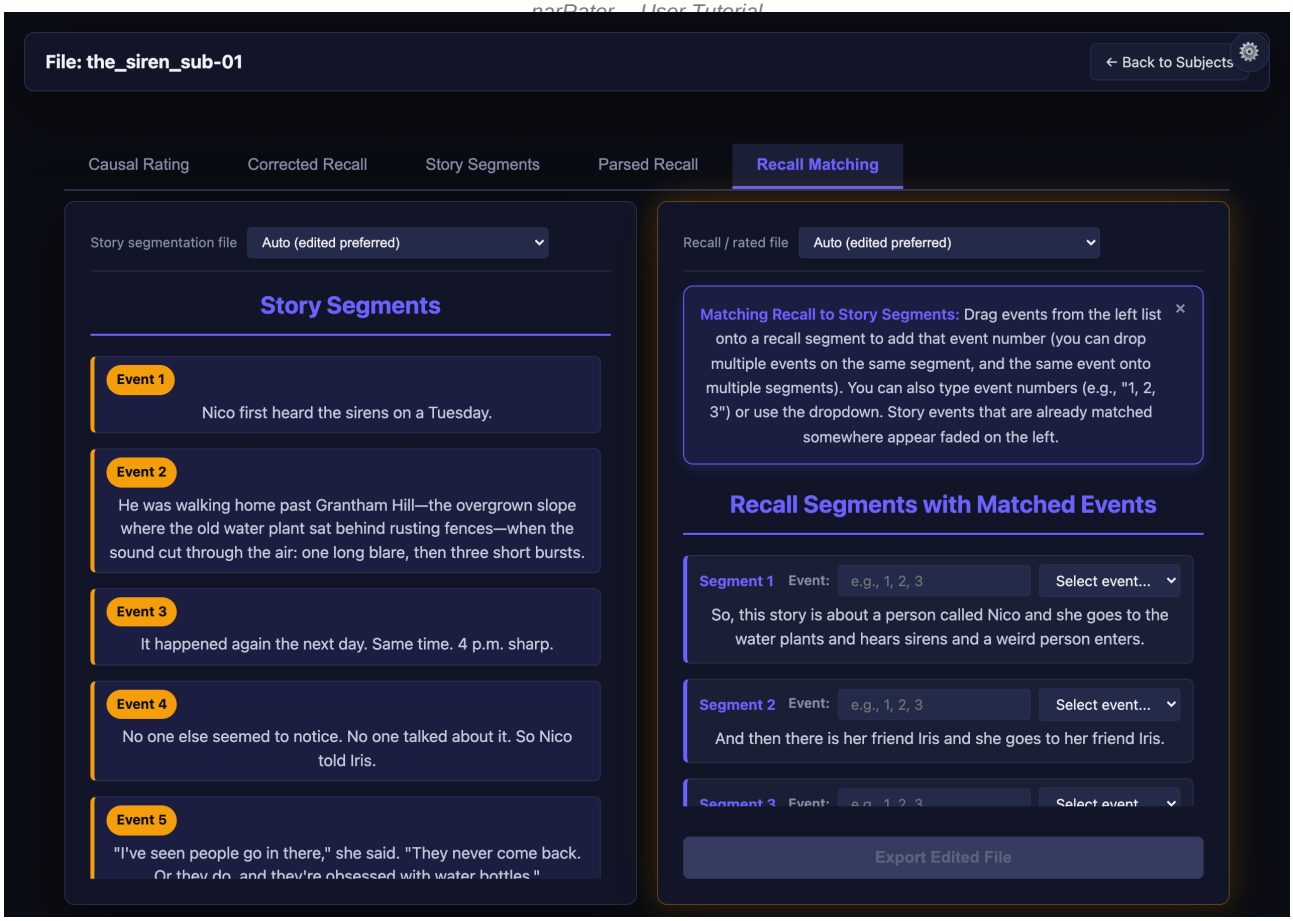
- RMATCH_MODEL_NAME -- HF model id (default google/gemma-4-31B-it)
- RMATCH_QUANTIZATION -- 4bit | 8bit | (empty for none)
- RMATCH_WINDOW_SIZE -- sliding-window size, integer (default 5)
- RMATCH_PROMPT -- one of primary, primary_no_story, primary_no_cot, primary_no_story_no_cot (default), secondary
- RMATCH_BATCH_SIZE -- inference batch size; defaults to 1 on macOS to avoid Metal NDArray limits
- RMATCH_FORCE_CPU -- 1/true to bypass Apple MPS (more RAM, slower)
- RMATCH_MAX_NEW_TOKENS -- per-call generation budget

WARNING: the default model google/gemma-4-31B-it is roughly 18 GB on disk and needs around 16 GB RAM even with 4-bit quantization. On a CPU-only machine, matching 30+ recall segments can take hours. Pick a smaller HF model (e.g. google/gemma-2-2b-it or Qwen/Qwen2.5-1.5B-Instruct) for tractable runtimes on a laptop, trading off accuracy. The launcher description repeats this warning.

Output filename suffix: -rmatch-pypi-{model-slug}-{quant}.xlsx so multiple rMatch runs with different models coexist without overwriting each other.

10.6 Interface Walkthrough

In the detail view, the Recall Matching tab shows the story events on the left and the recall segments on the right. Each segment has an editable field for the matched event number(s). Enter event numbers separated by commas for multiple matches.



Subject inspection -- Recall Matching tab (Step 5).

Example: Recall rating output ({subject_id}_rate-recall.xlsx)

recalled_events	recall_in_temporal_order
1	There was a girl named Clara.
2	She lived in a mountain town.
3	She went on an adventure one day.

10.7 Methods & Options Reference

Method	What it does	Args / Keys
test-mode (default)	Keyword/phrase overlap matcher. Fast, deterministic, no LLM.	`no key needed`
api	Anthropic batch matcher. Picks Anthropic model from dropdown.	`ANTHROPIC_API_KEY` `RECALL_RATING_MODEL=<id>`

gemma-ollama	Local Gemma 4 E4B via Ollama; JSON-schema constrained output.	`Ollama running locally`
		`RECALL_RATING_BACKEND=ollama`
		`OLLAMA_GEMMA_TAG override optional`

rmatch	PyPI rmatch + Hugging Face causal-LM. See Section 10.5 for size warnings.	`HF_TOKEN`
		`RECALL_RATING_BACKEND=rmatch`
		`RMATCH_MODEL_NAME, RMATCH_QUANTIZATION, RMATCH_WINDOW_SIZE, RMATCH_PROMPT`

manual	Copies parsed segments with empty event-match fields for hand-rating in UI.	`no key needed`

10.8 Terminal Command

Run Step 5 from the terminal:

```
narraters match --test-mode                # keyword, no model/API
narraters match --method api --story-events data/3_story_events
narraters match --method gemma-ollama
narraters match --method rmatch            # needs the [match] extra
```

--method: test | api | gemma-ollama | rmatch (--test-mode == --method test). --story-events points at the {story}_events.xlsx directory; -i/--input is the recall-parsed directory and -o/--output the rated-output directory.

11 Step 6: Causal Rating

11.1 What This Step Does

Rates the strength of the causal relation between every pair of story events. Operates on the segmented story events from Step 2 (not on the recall) and produces a square cause-by-effect matrix of ratings 0-3. Optional per-pair fields capture reasoning text, causal type (physical / motivational / psychological / enabling), and a semantic-similarity score.

11.2 How It Works

Input: data/3_story_events/{story}_events.xlsx (or a per-variant version)

Output: output/causal_rated/{story}_causal-{method}.xlsx (one row per (event_A, event_B) pair with rating > 0; optional reasoning / causal_type / semantic columns)

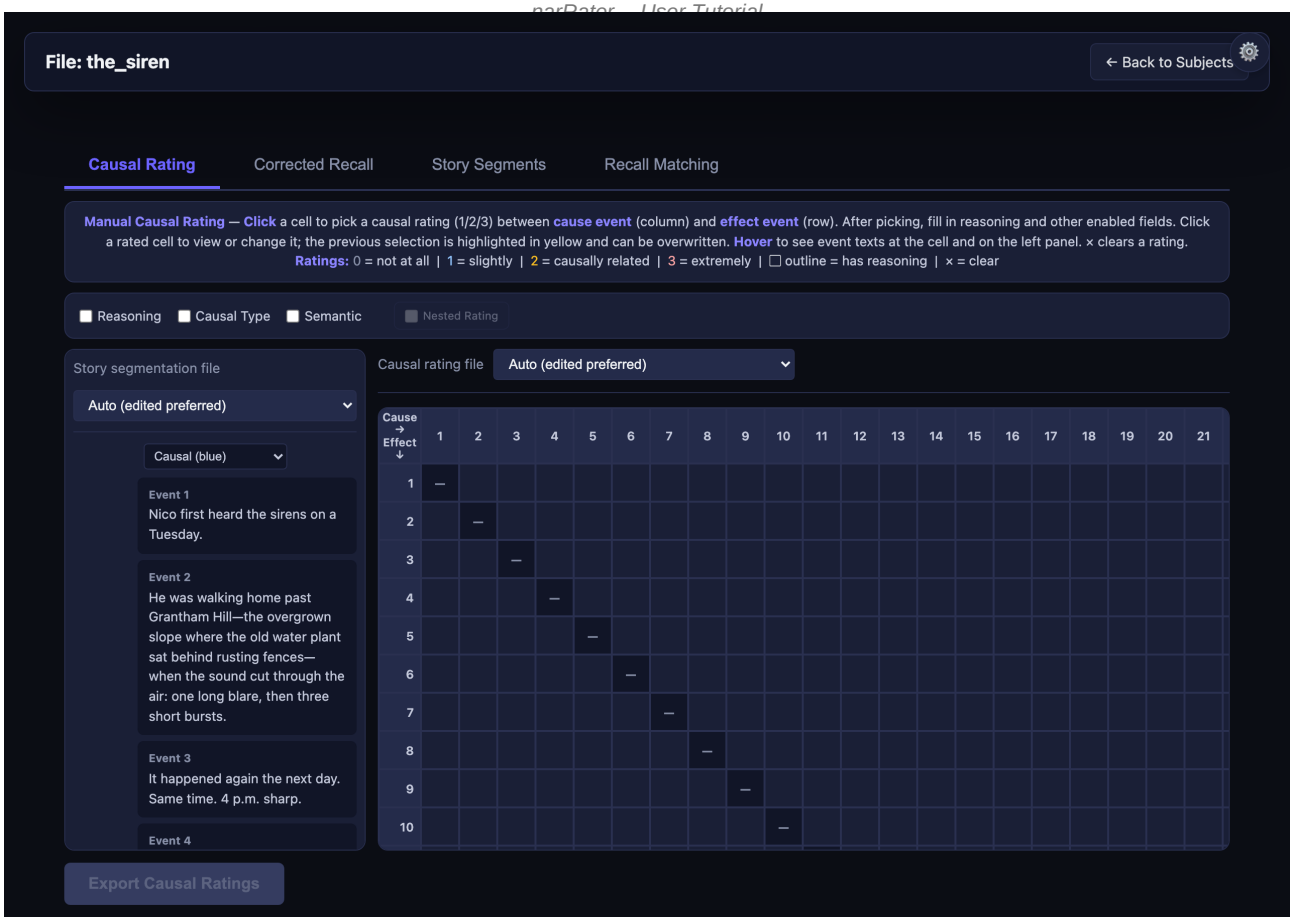
Available methods:

- Linguistic (default): Rule-based causal scoring using causal connectives, entity continuity, and action-consequence verb patterns. No API key required. Uses spaCy if installed; falls back to a regex-based scorer otherwise.
- API (LLM Call): Sends each pair (or batch of pairs) to an Anthropic or OpenAI model and parses a structured rating + reasoning. Model dropdown lists the same Anthropic options as Steps 2 and 5 (Opus 4.7, Sonnet 4.6, Haiku 4.5, Sonnet 4.5, Haiku 3.5; see Section 15.1). Requires an API key.
- Manual: Creates an empty causal-rating workbook so you can fill in ratings cell-by-cell in the matrix UI.

11.3 Cell-Picker UI

The detail view shows the cause-by-effect matrix. Hover any cell to preview both event texts; click a cell to open the multi-step picker. The picker asks for the rating (0-3), then -- depending on which flags are checked in the toolbar -- for reasoning text, causal type, and a semantic-similarity score. The currently saved value is highlighted in yellow so you can see it before overwriting. The mode dropdown above the matrix lets you visualize the matrix as Causal (default) or Semantic-similarity connection lines.

Click x in a cell (or pick 0) to clear a rating.



Causal Rating inspection tab on a story row: matrix + event sidebar. Regenerate screenshots with developer/capture_tutorial_screenshots.py.

11.4 Export With Custom Filename

The Export Causal Ratings button opens a popup pre-filled with a default path (output/causal_rated/{story}_causal-manual_{raternname}-edit.xlsx for manual edits, or the source file's name preserved for re-export). You can edit the filename in the popup before clicking Save; the file is written exactly to the path you enter, as long as it resolves to somewhere inside the project root. This is how you save named variants such as ..._trial2.xlsx or ..._post-review.xlsx.

Cancel closes the popup without saving. Enter confirms; Escape cancels.

11.5 Saved Columns

Each row in the output workbook is one rated (event_A, event_B) pair. Columns are gated by the flag checkboxes in the toolbar:

- event_A, event_B -- the cause and effect event numbers
- rating -- integer 1, 2, or 3 (rows with 0 are not saved)
- reasoning -- free text (only when the Reasoning flag is on)
- causal_type -- physical / motivational / psychological / enabling / other (only when the Causal Type flag is on)
- semantic -- 0-3 score (only when the Semantic flag is on)

11.6 Methods & Options Reference

Method	What it does	Args / Keys

linguistic (default)

Rule-based causal scorer using connectives, entity continuity, action-consequence verbs.

		no key needed spaCy used if installed
api	LLM-based causal rating. Pick Anthropic model from dropdown.	ANTHROPIC_API_KEY or OPENAI_API_KEY CAUSAL_RATING_MODEL=<id> CAUSAL_RATING_PROMPT=<name>
manual	Creates an empty rating workbook for hand-filling in the matrix UI.	no key needed

11.7 Terminal Command

Run Step 6 from the terminal:

```
narraters rate --method linguistic
narraters rate --method api --model claude-sonnet-4-6 \
               --prompt-version causal_rating
narraters rate --method manual     # empty NxN matrix to fill by hand
```

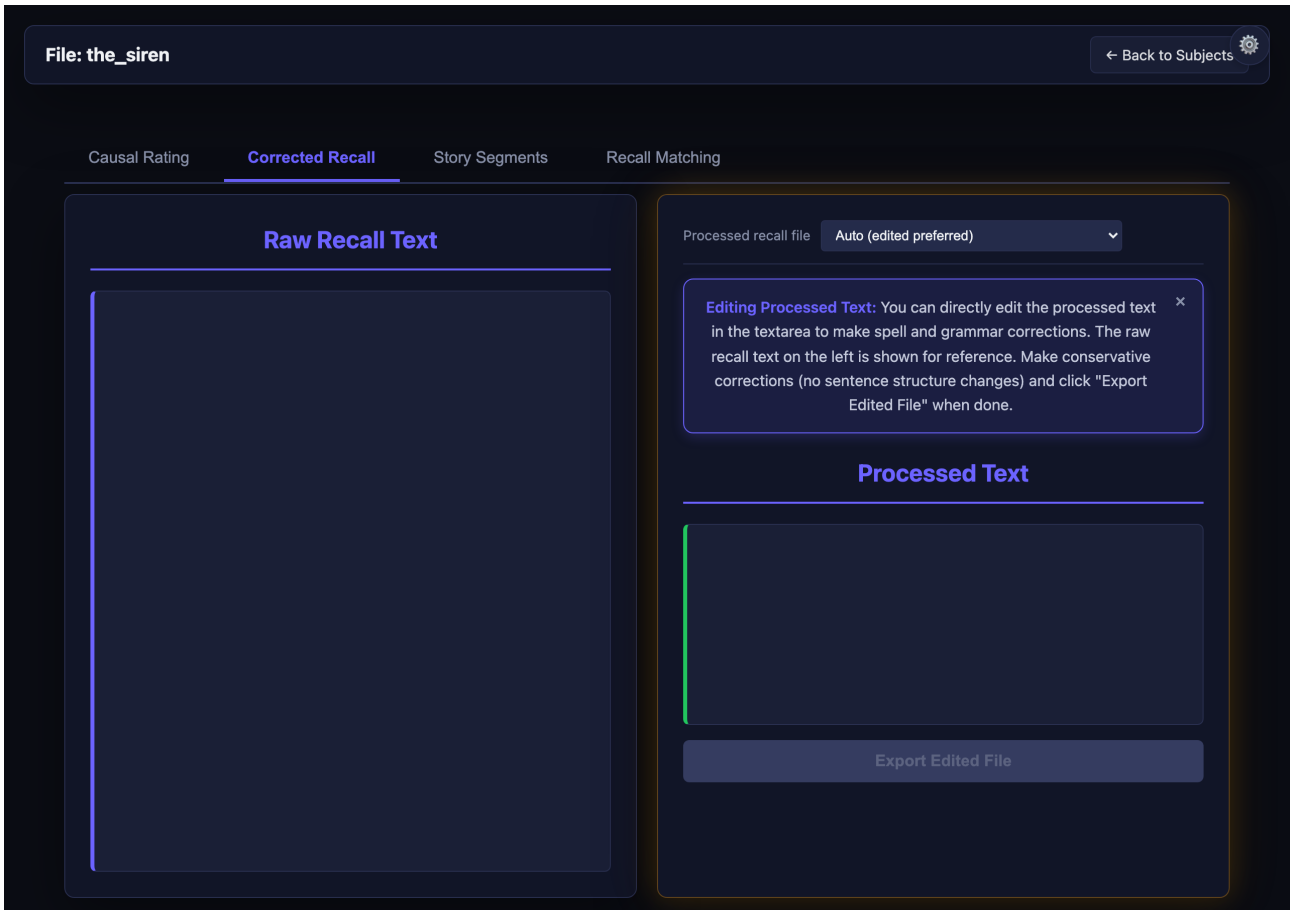
--method: linguistic (rule-based, no model) | api (LLM) | manual (writes an empty matrix for hand rating). --model and --prompt-version apply to --method api; -i/--input and -o/--output set the event-list input and causal-rated output.

12 Detail View and Manual Editing

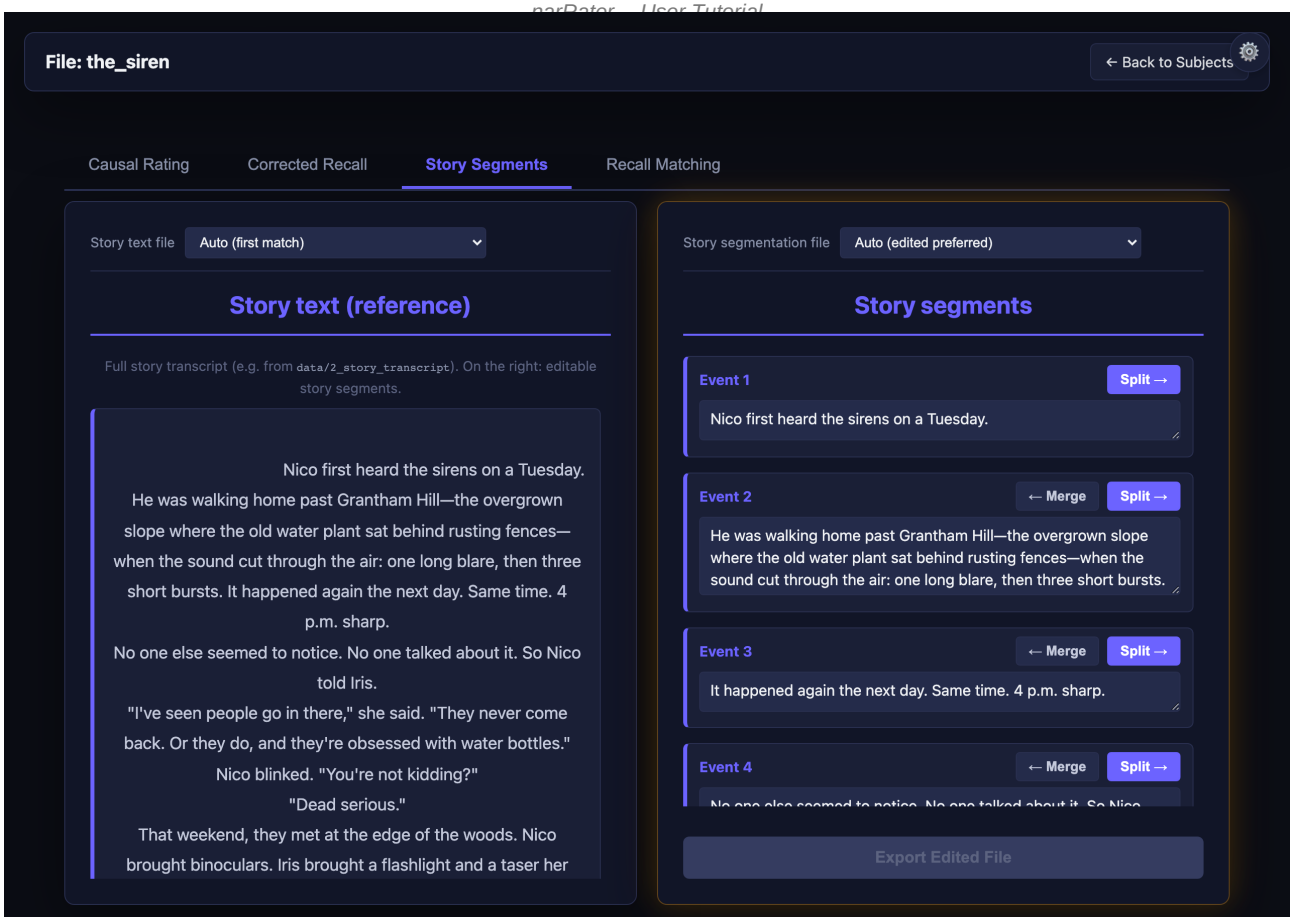
Click any row in the dashboard to open detail / inspection views. Stories and subjects each get a tab strip for whichever pipeline steps apply. Most recall tabs mirror the pipeline order shown on the Dashboard; story rows add Story Segments, optional story audio transcription, and -- when causalRating is configured -- Causal Rating.

Figures below are regenerated together from developer/capture_tutorial_screenshots.py using bundled the_siren/pieman examples; missing thumbnails mean screenshots were not rebuilt after a UI update.

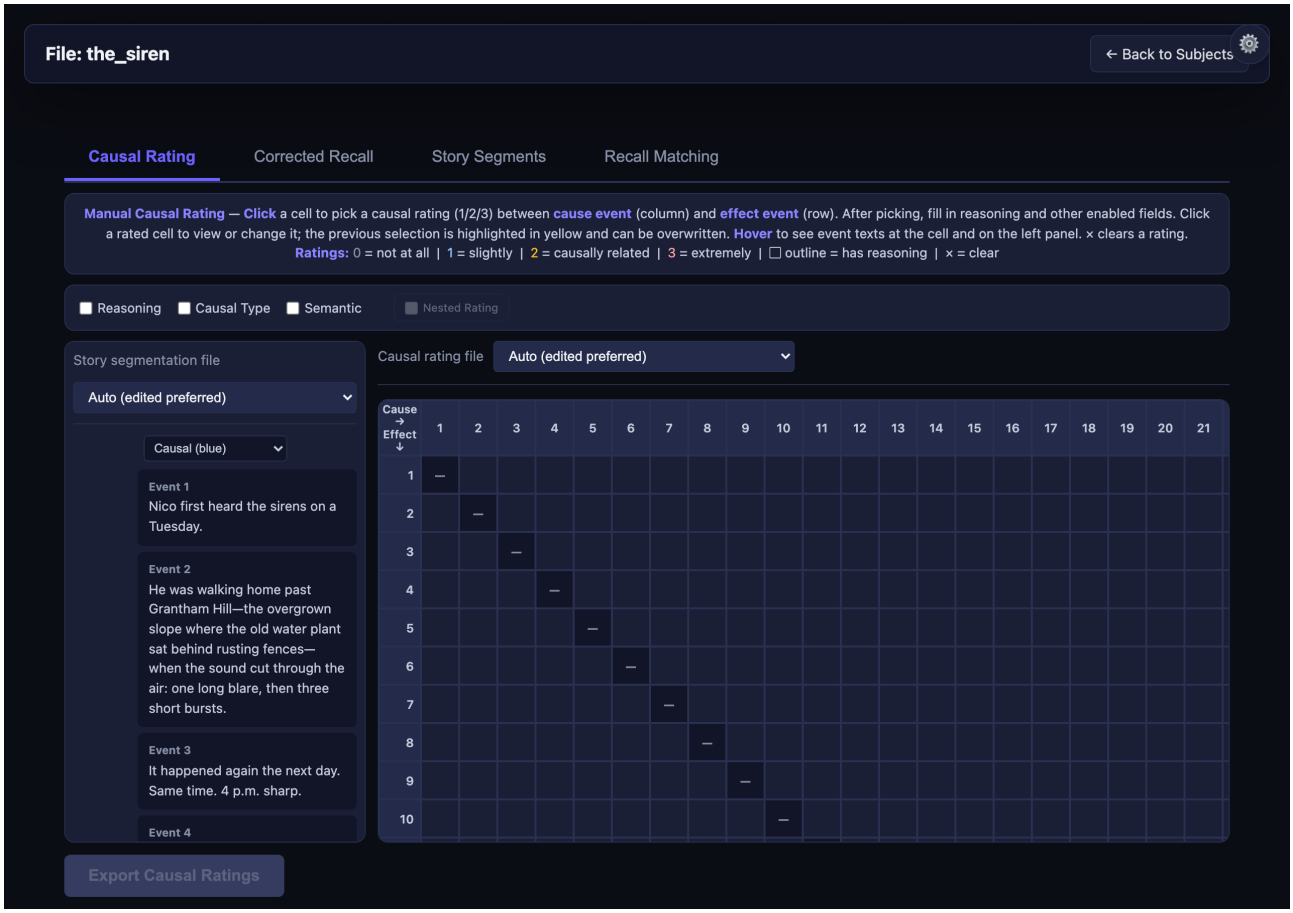
Story inspection snapshots



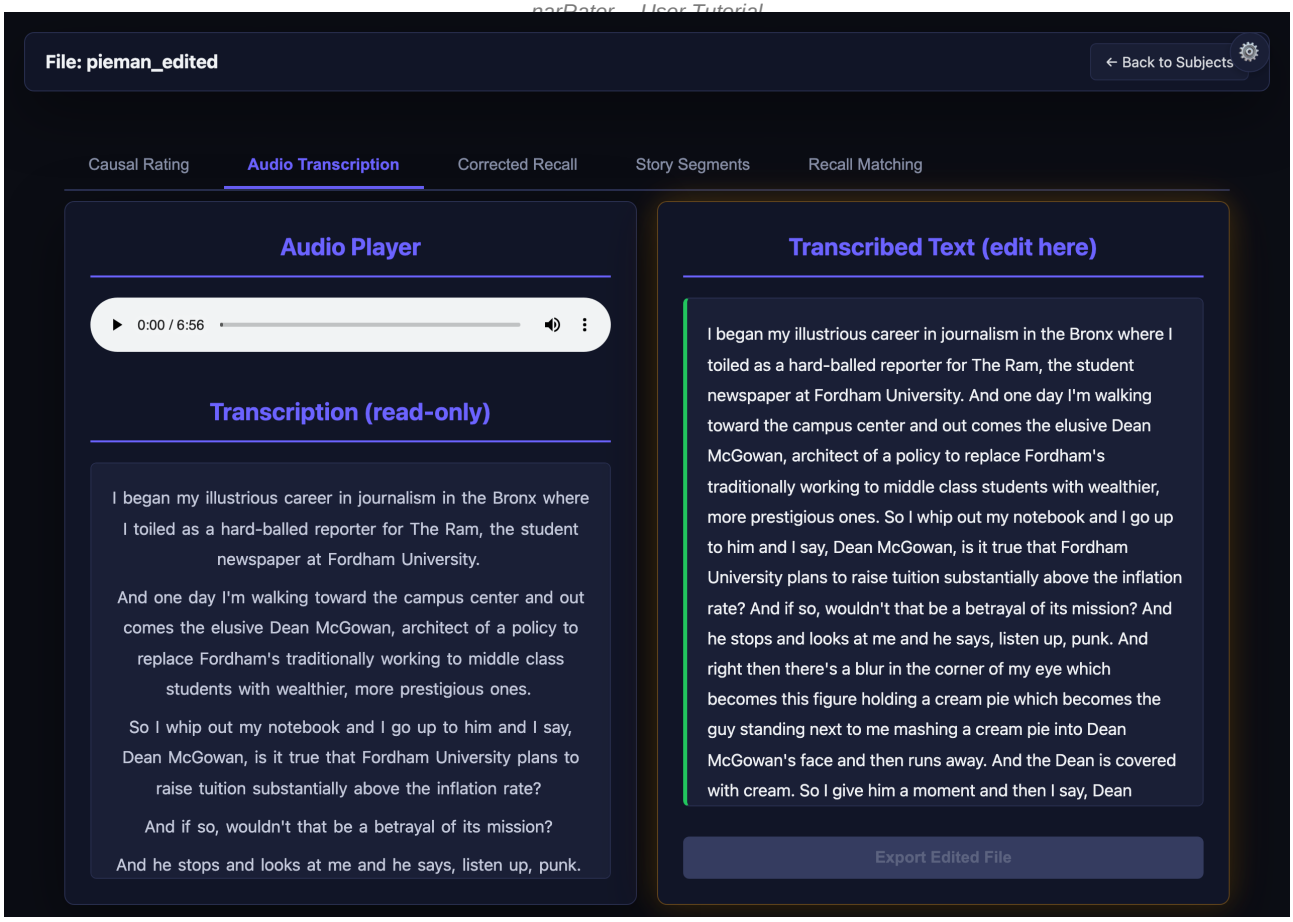
Story inspection (default routing) showing the story-row tab strip (example: bundled the_siren materials).



Story inspection deep-linked to segmented events / Story Segments layout (routing .../story/<name>/step0).

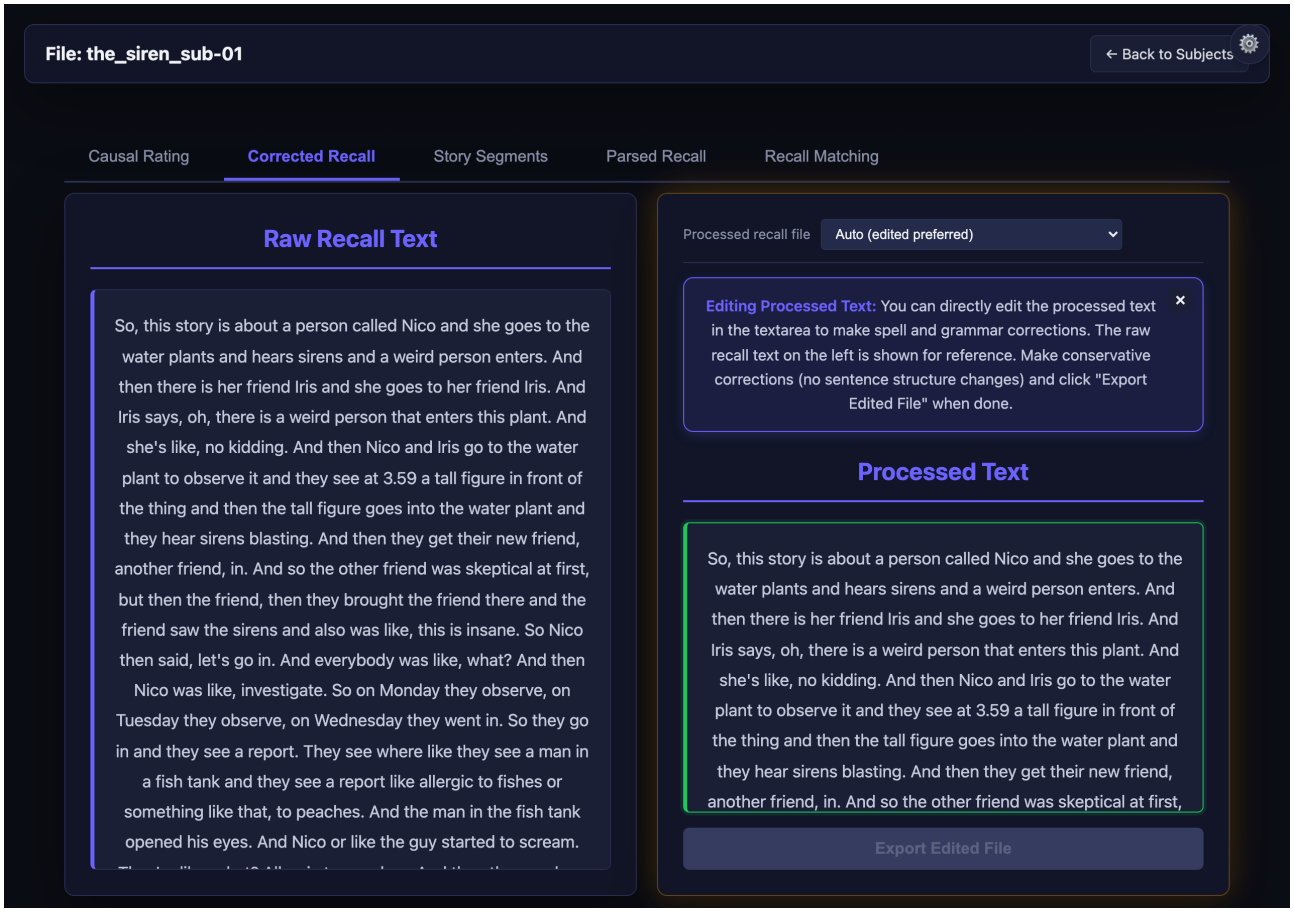


Causal Rating matrix mounted on story inspection (.../story/<name>/step0_causal when causalRating runs).

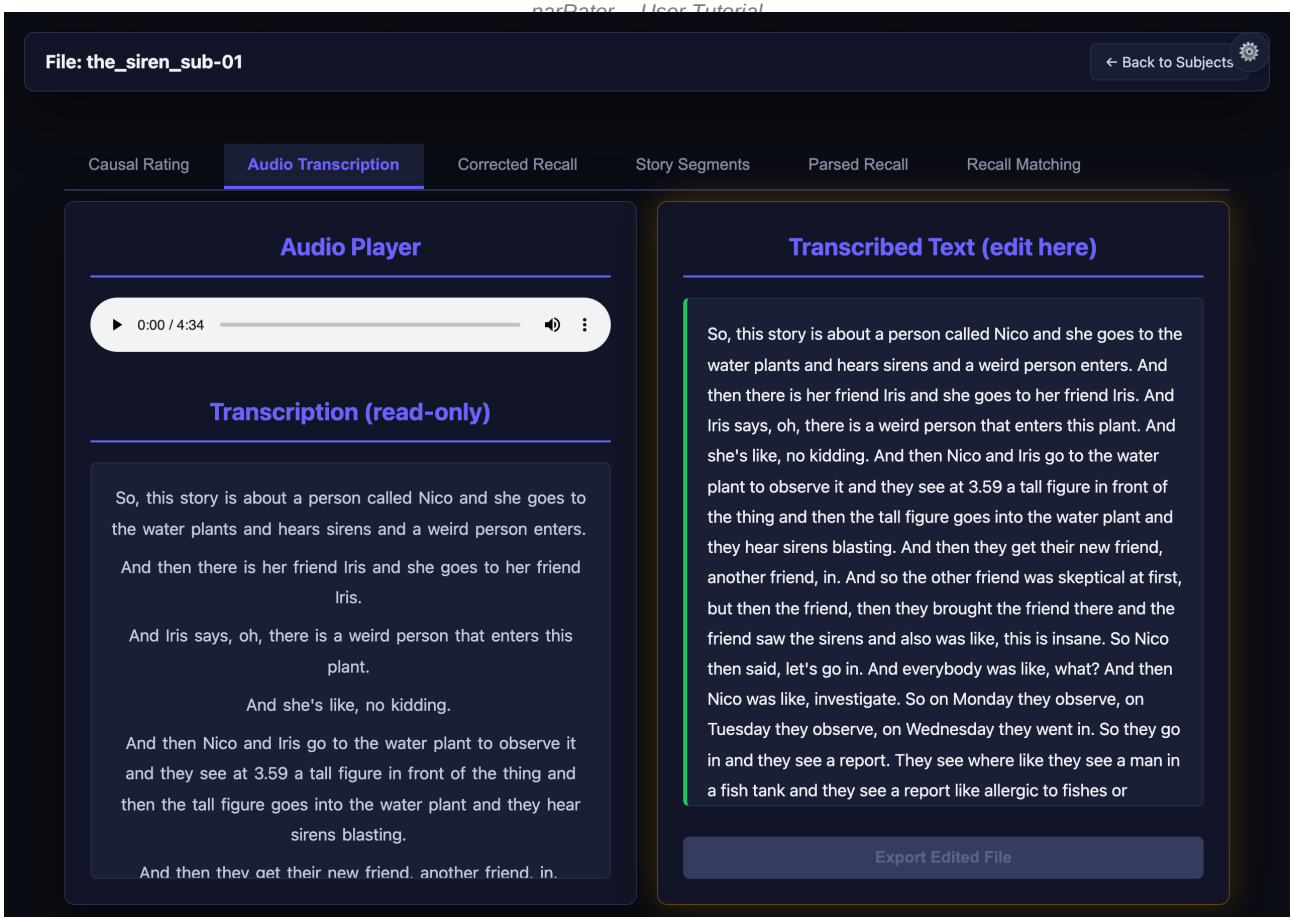


Story-level Audio Transcription tab (example capture with pieman_edited story audio bundled in data/).

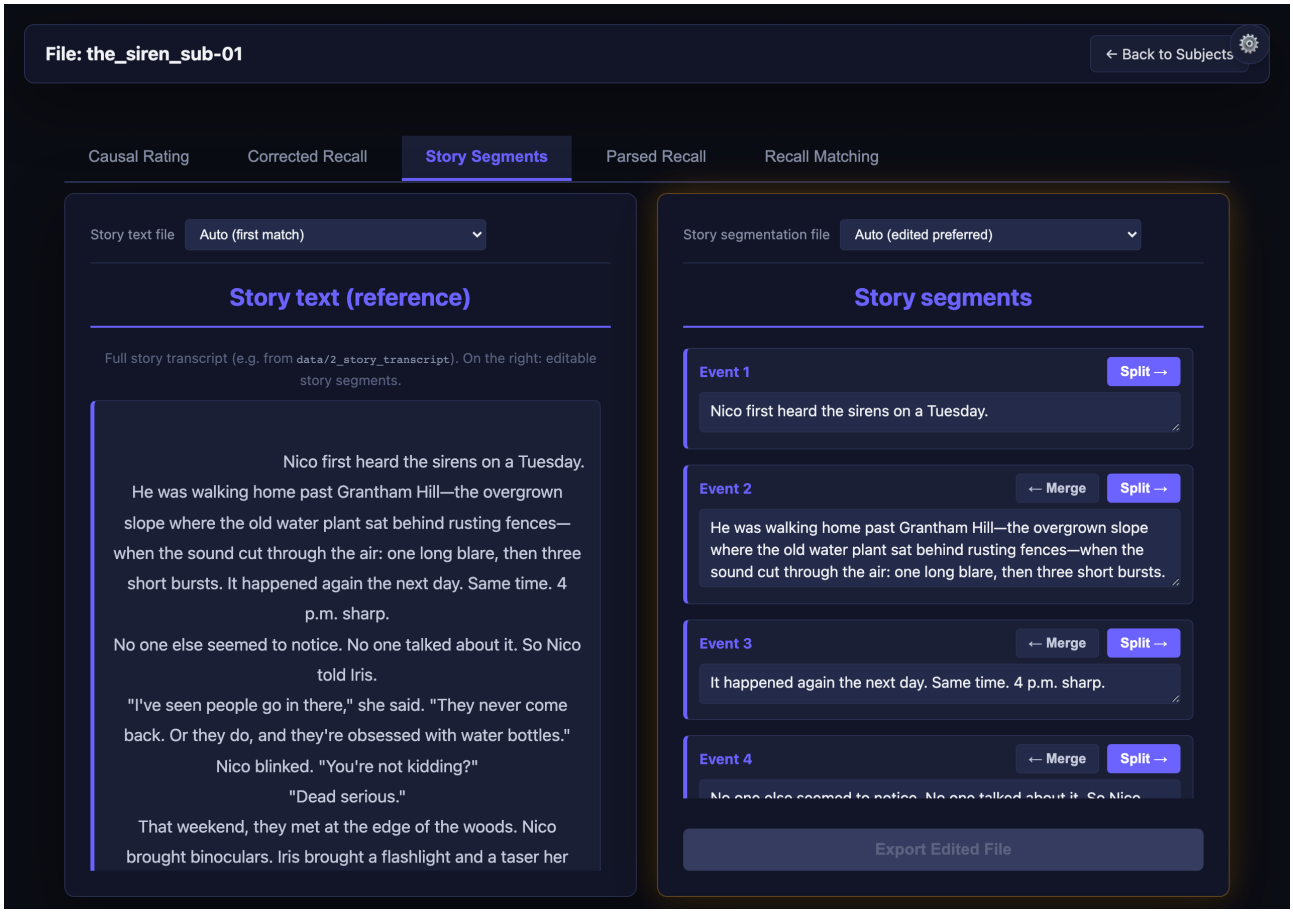
Subject recall inspection snapshots



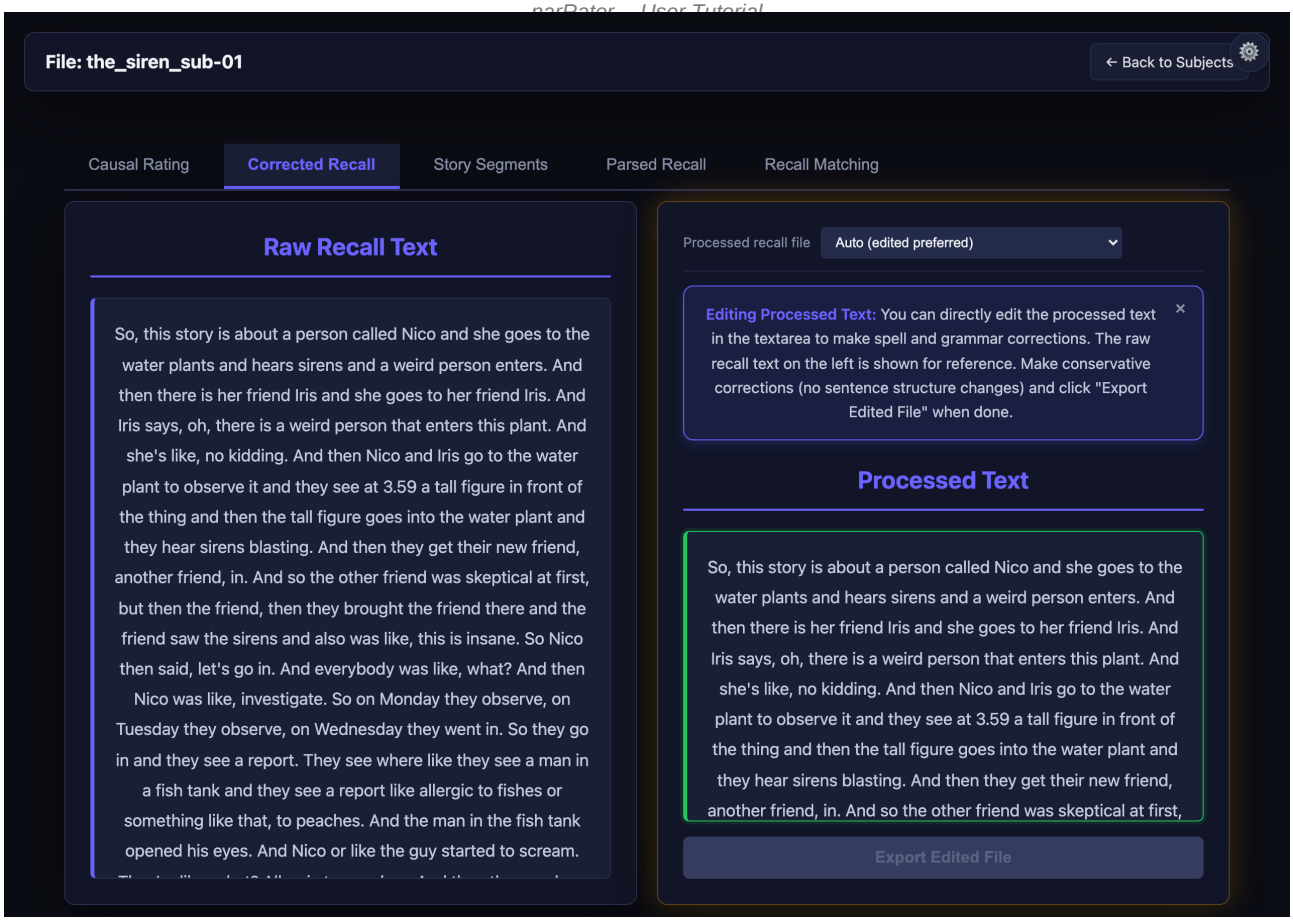
Subject inspection landing tab immediately after navigating from the dashboard (defaults depend on configured steps/data).



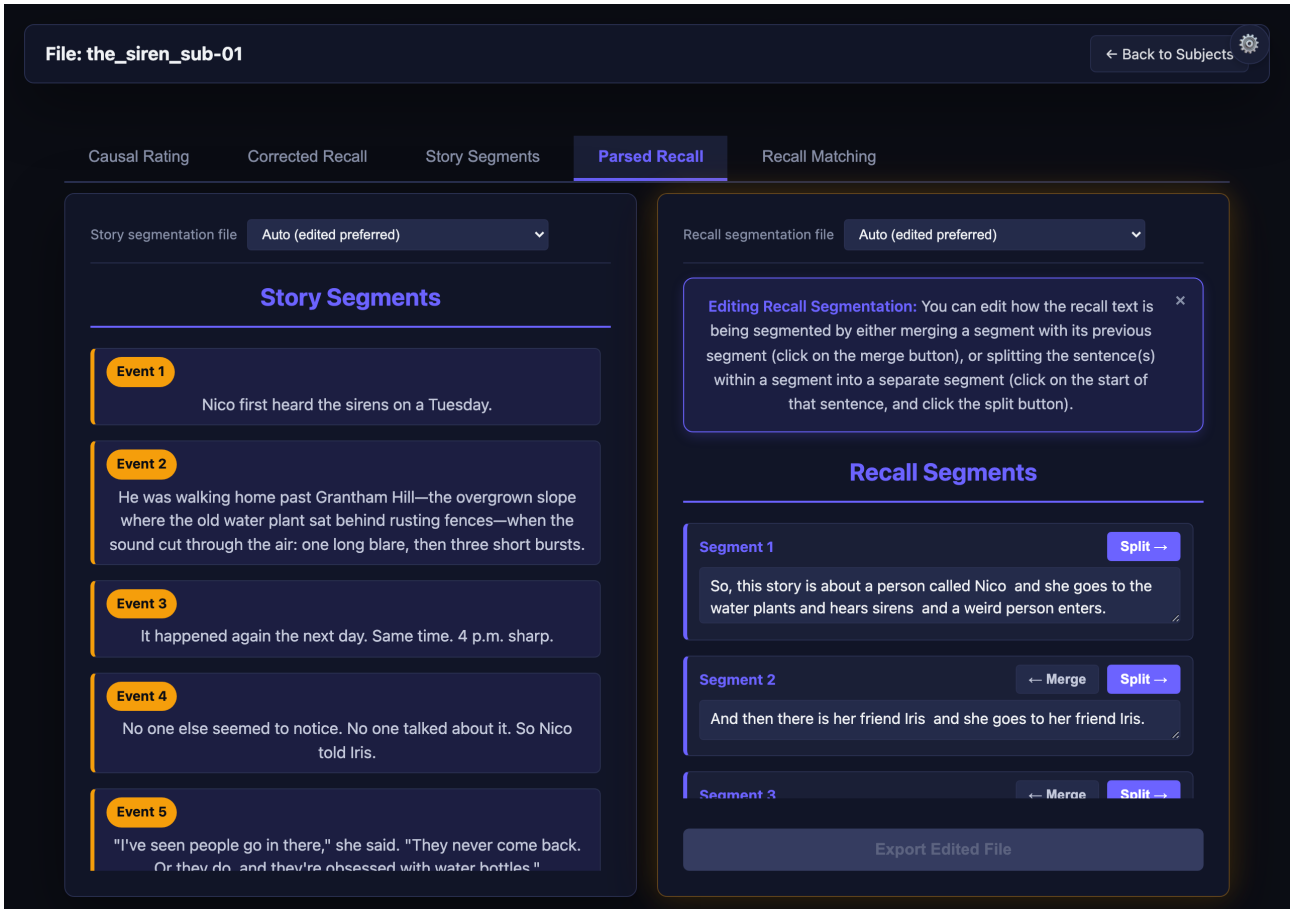
Recall inspection -- Audio Transcription tab for subjects with recall recordings (bundled mp4/audio for the_siren).



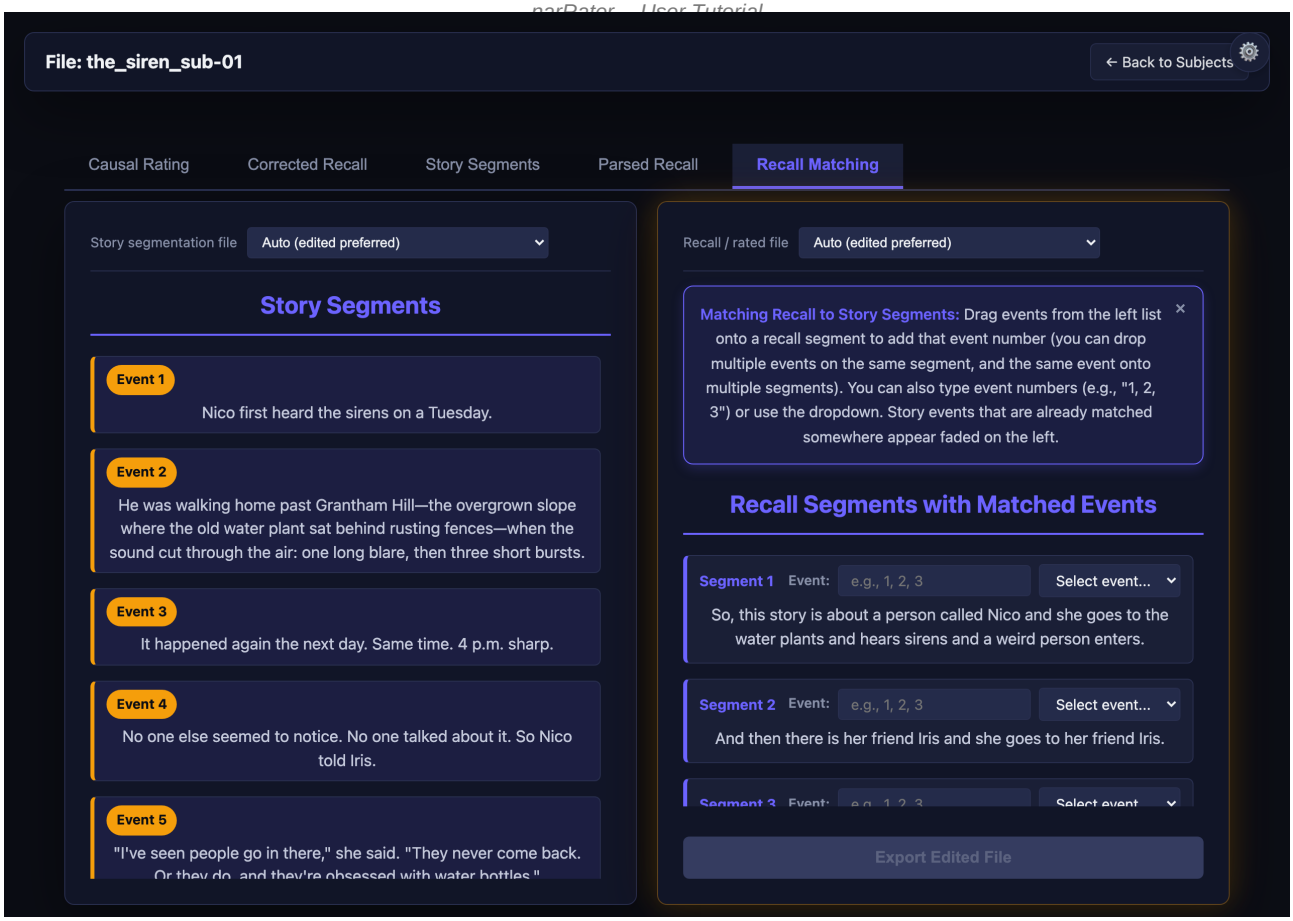
Subject inspection -- Story Segments reference tab injected whenever recall parsing steps need contextual story segmentation.



Subject inspection -- Corrected Recall.



Subject inspection -- Parsed Recall.



Subject inspection -- Recall Matching.

Editing controls per tab:

- Audio Transcription: audio player (left) + editable transcript (right). Export to save.
- Story Segments: full story transcript (left) + event segments (right). Split, Merge, Export.
- Corrected Recall: raw recall text (left) + corrected text editor (right). Export to save.
- Parsed Recall: story events reference (left) + parsed segments (right). Split, Merge, Export.
- Recall Matching: story events reference (left) + recall with event editors (right). Export to save.

When you export an edited file, it is saved with your rater name in the filename (e.g., {subject_id}_{ratername}-edit.txt). This keeps automated and manual versions separate. Use the File Version dropdown to switch between versions.

Keyboard shortcut: Ctrl+S (Windows/Linux) or Cmd+S (macOS) to save the current tab.

13 Output Files and Formats

13.1 Corrected Recall

Location: output/recall_corrected/

Format: Plain text (.txt)

Naming: {subject_id}.txt (auto) or {subject_id}_{ratername}-edit.txt (manual)

Example: Corrected recall file

```
There was a girl named Clara who lived in a mountain town.
She went on an adventure one day and met a wise old man.
```

13.2 Parsed Recall

Location: output/recall_parsed/

Format: Excel (.xlsx)

Columns: 'recalled_events' (initially empty), 'recall_in_temporal_order' (segment text)

13.3 Rated Recall

Location: output/recall_rated/

Format: Excel (.xlsx)

Columns: 'recalled_events' (matched event numbers), 'recall_in_temporal_order' (segment text)

13.4 Story Events

Location: data/3_story_events/

Format: Excel (.xlsx)

Columns: 'event' (number), 'story_texts' (segment text)

14 Command-Line Usage

You can also run each step directly from the terminal:

```
# Audio transcription
python scripts/1_audio-transcribe.py

# Story event segmentation (fine-grained)
python scripts/2_story-event-segment.py --method fine

# Story event segmentation (API with specific model)
python scripts/2_story-event-segment.py --method api --model gpt-4o

# List available models and prompts
python scripts/2_story-event-segment.py --list-models
python scripts/2_story-event-segment.py --list-prompts

# Spell & grammar correction
python scripts/3_spell-grammar-correct.py

# Recall parsing
python scripts/4_parse-texts.py

# Recall rating
python scripts/5_recall-rater.py
```

All outputs are saved to the output/ directory.

Common command-line options for Step 2 (Story Event Segmentation):

- --method {clause|fine|coarse|api}: choose segmentation method
- --model {model_name}: specify LLM model for API method
- --prompt-version {filename}: specify prompt file for API method
- --input {path}: specify input file instead of scanning the default directory
- --reconstruct {events.xlsx}: reconstruct story text from event segmentation file

15 API Configuration Reference

15.1 Supported Models

The following LLM models are exposed in the web-interface dropdown for steps that offer an API method (Step 2 Story Event Segmentation, Step 5 Recall Rating, and Step 6 Causal Rating).

Anthropic Messages API:

- claude-haiku-3-5-20241022 -- Haiku 3.5 (Anthropic)
- claude-haiku-4-5-20251001 -- Haiku 4.5 (Anthropic)
- claude-opus-4-7 -- Opus 4.7 (Anthropic)
- claude-sonnet-4-5-20250929 -- Sonnet 4.5 (Anthropic)
- claude-sonnet-4-6 -- Sonnet 4.6 (Anthropic)

OpenAI:

- gpt-4o -- high quality, alternative to Anthropic balanced tiers
- gpt-4o-mini -- faster and cheaper

Local (no cloud key):

- Ollama (Gemma 4 E4B) -- shared backend for Steps 3, 4, and 5; uses local Ollama server
- rMatch (Hugging Face) -- Step 5 only; PyPI rmatch package, requires HF_TOKEN

Model selection: pick from the API method dropdown in the launcher, or pass --model on the command line. The Step 5 launcher also exposes rMatch as a fourth method (Test Mode / API / Ollama / rMatch). Different steps may benefit from different models.

15.2 Prompt Files

All prompts are stored as plain text files in the scripts/prompt/ folder:

- scripts/prompt/event_segment.txt -- used by Step 2 (Story Event Segmentation) API method
- scripts/prompt/recall_rating.txt -- used by Step 5 (Recall Rating) API method

You can create alternative prompt versions by adding new files to the scripts/prompt/ folder (e.g., recall_rating_v2.txt). Select them via environment variable or command-line flag.

15.3 Setting Up API Keys

Three ways to provide API keys:

- Environment variable: export ANTHROPIC_API_KEY='sk-ant-...' (or OPENAI_API_KEY)
- Setup script: bash scripts/setup_api_key.sh (interactive prompts)
- Web interface: enter the key on the Pipeline Configuration page

API keys are never stored in code files. They are read from the environment at runtime.

16 Tips and Best Practices

- Always verify automated output before proceeding to the next step.
- Use Fine-Grained segmentation for most research purposes; switch to Clause Level only when you need the finest possible granularity.
- The web interface supports keyboard shortcuts: Ctrl+S / Cmd+S to save the current tab.
- Install spaCy for more accurate clause boundary detection: pip install spacy && python -m spacy download en_core_web_sm
- If you have an Anthropic API key, the API segmentation method and Anthropic-based recall rating tend to produce higher-quality results.
- Multiple users can work on the same dataset -- each rater's edits are saved with their rater name, so nothing is overwritten.
- For large datasets, use the Batch Process button on the dashboard to process all items at once.
- Check scripts/prompt/README.md for documentation on different recall rating prompt versions and their characteristics.